

Published in final edited form as:

Nat Comput. 2012 ; 7433: 25–42. doi:10.1007/s11047-013-9393-6.

Deterministic Function Computation with Chemical Reaction Networks*

Ho-Lin Chen[†], David Doty[‡], and David Soloveichik[§]

Ho-Lin Chen: holinc@gmail.com; David Doty: ddoty@caltech.edu; David Soloveichik: david.soloveichik@ucsf.edu

[†]National Taiwan University, Taipei, Taiwan

[‡]California Institute of Technology, Pasadena, CA, USA

[§]University of California, San Francisco, San Francisco, CA, USA

Abstract

Chemical reaction networks (CRNs) formally model chemistry in a well-mixed solution. CRNs are widely used to describe information processing occurring in natural cellular regulatory networks, and with upcoming advances in synthetic biology, CRNs are a promising language for the design of artificial molecular control circuitry. Nonetheless, despite the widespread use of CRNs in the natural sciences, the range of computational behaviors exhibited by CRNs is not well understood.

CRNs have been shown to be efficiently Turing-universal (i.e., able to simulate arbitrary algorithms) when allowing for a small probability of error. CRNs that are guaranteed to converge on a correct answer, on the other hand, have been shown to decide only the semilinear predicates (a multi-dimensional generalization of “eventually periodic” sets). We introduce the notion of function, rather than predicate, computation by representing the output of a function $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ by a count of some molecular species, i.e., if the CRN starts with $x_1, \dots, x_k$ molecules of some “input” species $X_1, \dots, X_k$, the CRN is guaranteed to converge to having $f(x_1, \dots, x_k)$ molecules of the “output” species $Y_1, \dots, Y_l$. We show that a function $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ is deterministically computed by a CRN if and only if its graph $\{(\mathbf{x}, \mathbf{y}) \in \mathbb{N}^k \times \mathbb{N}^l \mid f(\mathbf{x}) = \mathbf{y}\}$ is a semilinear set.

Finally, we show that each semilinear function f (a function whose graph is a semilinear set) can be computed by a CRN on input $\mathbf{x}$ in expected time $O(\text{polylog } \|\mathbf{x}\|_1)$.

1 Introduction

The engineering of complex artificial molecular systems will require a sophisticated understanding of how to *program chemistry*. A natural language for describing the interactions of molecular species in a well-mixed solution is that of (finite) chemical reaction networks (CRNs), i.e., finite sets of chemical reactions such as $A + B \rightarrow A + C$. When the behavior of individual molecules is modeled, CRNs are assigned semantics

*A preliminary version of this paper appeared as [8]. The first author was supported by the Molecular Programming Project under NSF grant 0832824, the second and third authors were supported by a Computing Innovation Fellowship under NSF grant 1019343. The second author was supported by NSF grants CCF-1219274 and CCF-1162589. The third author was supported by NIGMS Systems Biology Center grant P50 GM081879.

through *stochastic chemical kinetics* [13], in which reactions occur probabilistically with rate proportional to the product of the molecular count of their reactants and inversely proportional to the volume of the reaction vessel.

Traditionally CRNs have been used as a descriptive language to analyze naturally occurring chemical reactions (as well as numerous other systems with a large number of interacting components such as gene regulatory networks and animal populations). However, recent investigations have viewed CRNs as a programming language for engineering artificial systems. These works have shown CRNs to have eclectic computational abilities. Researchers have investigated the power of CRNs to simulate Boolean circuits [18], neural networks [14], and digital signal processing [15]. CRNs can simulate a bounded-space Turing machine efficiently, if the number of reactions is allowed to scale polynomially with the Turing machine's space usage [26]. Other work has shown CRNs can efficiently simulate a bounded-space Turing machine, with the number of reactions independent of the space bound, albeit with an arbitrarily small, non-zero probability of error [3].¹ Even Turing universal computation is possible with an arbitrarily small, non-zero probability of error over all time [24]. The computational power of CRNs also provides insight on why it can be computationally difficult to simulate them [23], and why certain questions are frustratingly difficult to answer (or even undecidable)[11, 27]. For example, it is PSPACE-hard to predict whether a particular species is producible [26]. The programming approach to CRNs has also, in turn, resulted in novel insights regarding natural cellular regulatory networks [7]. The importance of the model is underscored by the fact that equivalent models repeatedly arise in theoretical computer science under different guises: e.g. vector addition systems [16], petri nets [20], population protocols [1].

Recent work proposes concrete chemical implementations of arbitrary CRNs, particularly using nucleic-acid strand-displacement cascades as the physical reaction primitive [25, 6]. Thus, since in principle any CRN can be built, hypothetical CRNs with interesting behaviors are becoming of more than theoretical interest. One day artificial CRNs may underlie embedded controllers for biochemical, nanotechnological, or medical applications, where environments are inherently incompatible with traditional electronic controllers.

One of the best-characterized computational abilities of CRNs is the deterministic computation of predicates (decision problems) as investigated by Angluin, Aspnes and Eisenstat [2]. (They considered the equivalent distributed computing model of *population protocols* motivated by sensor networks.) Some CRNs, when started in an initial configuration assigning nonnegative integer counts to each of k different input species, are guaranteed to converge on a single “true” or “false” answer, in the sense that there are two special “voting” species T and F so that eventually either T is present and F absent to indicate “true”, or vice versa to indicate “false.” The set of inputs $S \subseteq \mathbb{N}^k$ that cause the system to answer “true” is then a representation of the decision problem solved by the CRN. Angluin, Aspnes and Eisenstat showed that the input sets S decidable by some CRN are precisely the *semilinear* subsets of $\mathbb{N}^k$ (see below).

¹This is surprising since finite CRNs necessarily must represent large binary data strings in a unary encoding, since they lack positional information to tell the difference between two molecules of the same species.

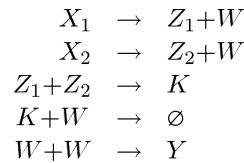
We extend these prior investigations of decision problems or predicate computation to study deterministic *function* computation. Consider the three examples in Fig. 1(top). These CRNs have the property that they converge to the right answer no matter the order in which the reactions happen to occur, and are thus insensitive to stochastic effects as well as reaction rate constants. Formally, we say a function $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ is computed by a CRN C if the following is true. There are “input” species $X_1, \dots, X_k$ and “output” species $Y_1, \dots, Y_l$ such that, if C is initialized with $x_1, \dots, x_k$ copies of $X_1, \dots, X_k$, then it is guaranteed to reach a configuration in which the counts of $Y_1, \dots, Y_l$ are described by the vector $f(x_1, \dots, x_k)$, and these counts never again change. For example, the CRN C with the single reaction $X \rightarrow 2Y$ computes the function $f(x) = 2x$ in the sense that, if C starts in an initial configuration with x copies of X and 0 copies of Y , then C is guaranteed to stabilize to a configuration with $2x$ copies of Y . Similarly, the function $f(x) = \lfloor x/2 \rfloor$ is computed by the single reaction $2X \rightarrow Y$ (Fig. 1(a)), in that the final configuration is guaranteed to have exactly $\lfloor x/2 \rfloor$ copies of Y (and 0 or 1 copies of X , depending on whether x is even or odd).

It is illuminating to compare the computation of division by 2 shown in Fig. 1(a) with another reasonable alternative: reactions $X \rightarrow Y$ and $Y \rightarrow X$ (i.e. the reversible reaction $X \rightleftharpoons Y$). If the rate constants of the two reactions are equal, the system equilibrium is at half of the initial amount of X transformed to Y . There are two stark differences between this implementation and that of Fig. 1(a). First, this CRN would not have an exact output count of Y , but rather a distribution around the equilibrium. (However, in the limit of large numbers, the error as a fraction of the total would converge to zero.) Second, the equilibrium amount of Y for any initial amount of X would depend on the relative rate constants of the two reactions. In contrast, the deterministic computation discussed in this paper relies on the identity and stoichiometry of the reactants and products rather than the rate constants. While the rates of reactions are analog quantities, the identity and stoichiometry of the reactants and products are naturally digital. Methods for physically implementing CRNs naturally yield systems with digital stoichiometry that can be set exactly [25, 6]. While rate constants can be tuned, being analog quantities, it cannot be expected that they can be controlled precisely.

A few general properties of this type of deterministic computation can be inferred. The first property is that a deterministic CRN is able to handle input molecules added at any time, and not just initially. Otherwise, if the CRN could reach a state after which it no longer “accepts input”, then there would be a sequence of reactions that would lead to an incorrect output even if all input is present initially. (It is always possible that some input molecules remain unreacted for arbitrarily long.)

The second general property of deterministic computation relates to composition. As any bona fide computation must be composable, it is important to ask: can the output of one deterministic CRN be the input to another? The problem is that deterministic CRNs have, in general, no way of knowing when they are done computing, or whether they will change their answer in the future. This is essentially because a CRN cannot deterministically detect the absence of a species, and thus, for example, cannot discern when all input has been read. Moreover, simply concatenating two deterministic CRNs (renaming species to avoid conflict) does not always yield a deterministic CRN. For example, consider computing the

function $f(x_1, x_2) = \lfloor \max(x_1, x_2)/2 \rfloor$ by composing the CRNs in Fig. 1(c) and (a). The new CRN is:



where W is the output species of the max computation, that acts as the input to the division by 2 computation. Note that if W happens to be converted to Y by the last reaction before it reacts with K , then the system can converge to a final output value of Y that is larger than expected. In other words, because the first CRN needs to consume its output W , the second CRN can interfere by consuming W itself (in the process of reading it out).

In contrast to the above example, two deterministic CRNs *can* be simply concatenated to make a new deterministic CRN if the first CRN never consumes its output species (i.e. it produces its output “monotonically”). Since it doesn’t matter when the input to the second CRN is produced (the first property, above), the overall computation will be correct. Yet deterministically computing a non-monotonic function without consuming output species is impossible (see Section 4). In a number of places in this paper, we convert a non-monotonic function into a monotonic one over more outputs, to allow the result to be used by a downstream CRN.

What do the functions in Fig. 1(top) have in common such that the CRNs computing them can inevitably progress to the right answer no matter what order the reactions occur in? What other functions can be computed similarly? Answering these questions may seem difficult because it appears like the three examples, although all deterministic, operate on different principles and seem to use different ideas.

We show that the functions deterministically computable by CRNs are precisely the semilinear functions, where we define a function to be *semilinear* if its graph $\{(\mathbf{x}, \mathbf{y}) \in \mathbb{N}^k \times \mathbb{N}^l \mid f(\mathbf{x}) = \mathbf{y}\}$ is a semilinear subset of $\mathbb{N}^k \times \mathbb{N}^l$. This means that the graph of the function is a union of a finite number of *linear* sets – i.e. sets that can be written in the form

$$\{\mathbf{b} + n_1 \mathbf{u}_1 + \dots + n_p \mathbf{u}_p \mid n_1, \dots, n_p \in \mathbb{N}\} \quad (1.1)$$

for some fixed vectors $\mathbf{b}, \mathbf{u}_1, \dots, \mathbf{u}_p \in \mathbb{N}^{k+l}$. Fig. 1 (bottom) shows the graphs of the three example functions expressed as a union of sets of this form. Informally, semilinear functions can be thought of as “piecewise linear functions” with a finite number of pieces, and linear domains of each piece.²

²Semilinear sets have a number of characterizations. They are often thought of as generalizations of arithmetic progressions. They are also exactly the sets that are definable in Presburger arithmetic [21]: the first-order theory of the natural numbers with addition. Equivalently, they are the sets accepted by boolean combinations of “modulo” and “threshold” predicates [2]. Semilinear functions are less well-studied. The “piecewise linear” intuitive characterization is formalized in Lemma 4.3.

This characterization implies, for example, that such functions as $f(x_1, x_2) = x_1x_2$, $f(x) = x^2$, or $f(x) = 2^x$ are not deterministically computable. For instance, the graph of the function $f(x_1, x_2) = x_1x_2$ consists of infinitely many lines of different slopes, and thus, while each line is a linear set, the graph is not a finite union of linear sets. Our result employs the predicate computation characterization of Angluin, Aspnes and Eisenstat [2], together with some nontrivial additional technical machinery.

While the example CRNs in Fig. 1 all seem to use different “tricks”, in Section 4 we develop a systematic construction for any semilinear function. To get the gist of this construction see the example in Fig. 2. To obtain a CRN computing the example semilinear function $f(x_1, x_2) = \max(2x_1 - x_2, x_2)$, we decompose the function into “linear” pieces: $f_1(x_1, x_2) = 2x_1 - x_2$ and $f_2(x_1, x_2) = x_2$ (formally partial affine functions, see Section 2). Then semilinear predicate computation (per [2]) is used to decide which linear function should be applied to a given input. A decomposition compatible with this approach is always possible by Lemma 4.3. Linear functions such as f_1 and f_2 are easy for CRNs to deterministically compute by the relative stoichiometry of the reactants and products (analogously to the example in Fig. 1(a)). However, note that to correctly compose the computation of f_1 with the downstream computation (Fig. 1(b), right column) we convert f_1 from a non-monotonic function with one output, to a monotonic function with two outputs such that the original output is encoded by their difference.

In the last part of this paper, we turn our attention to optimizing the time required for CRNs to converge to the answer. While the construction of Section 4 uses $O(\|\mathbf{x}\| \log \|\mathbf{x}\|)$ time, in Section 5, we show that every semilinear function can be deterministically computed on input $\mathbf{x}$ in expected time $\text{polylog}(\|\mathbf{x}\|)$. This is done by a similar technique used by Angluin, Aspnes, and Eisenstat [2] to show the equivalent result for predicate computation. They run a slow deterministic computation in parallel with a fast randomized computation, allowing the deterministic computation to compare the two answers and update the randomized answer only if it is incorrect, which happens with low probability. However, novel techniques are required since it is not as simple to “nondestructively compare” two integers (so that the counts are only changed if they are unequal) as to compare two Boolean values.

2 Preliminaries

Given a vector $\mathbf{x} \in \mathbb{N}^k$, let $\|\mathbf{x}\| = \|\mathbf{x}\|_1 = \sum_{i=1}^k \mathbf{x}(i)$, where $\mathbf{x}(i)$ denotes the i th coordinate of $\mathbf{x}$. We abuse notation and consider the sets $\mathbb{N}^k \times \mathbb{N}^l$ and $\mathbb{N}^{k+l}$ to be the same, because it is sometimes convenient to treat an ordered pair of vectors as being concatenated into a single longer vector. A set $A \subseteq \mathbb{N}^k$ is *linear* if there exist vectors $\mathbf{b}, \mathbf{u}_1, \dots, \mathbf{u}_p \in \mathbb{N}^k$ such that

$$A = \{\mathbf{b} + n_1 \mathbf{u}_1 + \dots + n_p \mathbf{u}_p \mid n_1, \dots, n_p \in \mathbb{N}\}.$$

Set A is *semilinear* if it is a finite union of linear sets. If $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ is a function, define the *graph* of f to be the set $\{(\mathbf{x}, \mathbf{y}) \in \mathbb{N}^k \times \mathbb{N}^l \mid f(\mathbf{x}) = \mathbf{y}\}$. A function is *semilinear* if its graph is a semilinear set.

We say a partial function $f: \mathbb{N}^k \dashrightarrow \mathbb{N}^l$ is *affine* if there exist kl rational numbers $a_{1,1}, \dots, a_{k,l} \in \mathbb{Q}$ and $l + k$ nonnegative integers $b_1, \dots, b_l, c_1, \dots, c_k \in \mathbb{N}$ such that, if $\mathbf{y} = f(\mathbf{x})$, then for

each $j \in \{1, \dots, l\}$, $y(j) = b_j + \sum_{i=1}^k a_{i,j}(\mathbf{x}(i) - c_i)$, and for each $i \in \{1, \dots, k\}$, $\mathbf{x}(i) - c_i \geq 0$. (In matrix notation, there exist a $k \times l$ rational matrix $\mathbf{A}$ and vectors $\mathbf{b} \in \mathbb{N}^l$ and $\mathbf{c} \in \mathbb{N}^k$ such that $f(\mathbf{x}) = \mathbf{A}(\mathbf{x} - \mathbf{c}) + \mathbf{b}$.) In other words, the graph of f , when projected onto the $(k+1)$ -dimensional space defined by the k coordinates corresponding to $\mathbf{x}$ and the single coordinate corresponding to $y(j)$, is a subset of a k -dimensional hyperplane.

Four aspects of the definition of affine functions invite explanation.

First, we allow partial functions because Lemma 4.3 characterizes the semilinear functions as finite combinations of affine functions, where the union of the domains of the functions is the entire input space $\mathbb{N}^k$. The value of an affine function on an input outside of its domain is irrelevant (and in fact may be non-integer).

Second, we have two separate “constant offsets” b_j and c_i . Affine functions over the reals are typically defined with only one of these, b_j . Our definition captures the form that is directly computable with molecular counts: we can take $\mathbf{x}$, subtract $\mathbf{c}$, multiply by $\mathbf{A}$, and subtract $\mathbf{b}$, with every intermediate result being integer-valued. If we try to incorporate the $\mathbf{c}$ offset into the $\mathbf{b}$ offset, we could end up with fractional intermediate computations.

Third, it may seem overly restrictive to require b_j and c_i to be nonnegative. In fact, our proof of Lemma 4.2 is easily modified to show how to construct a CRN to compute an affine function that allows negative values for b_j and c_i . However, Lemma 4.3 shows that, when the function is such that its graph is a nonnegative linear set, then we may freely assume that b_j and c_i to be nonnegative. Since this simplifies some of our definitions, we use this convention.

Fourth, the requirement that $\mathbf{x}(i) - c_i \geq 0$ seems artificial. When we prove that every semilinear function can be written as a finite union of partial affine functions with linear graphs (Lemma 4.3), however, this will follow from the fact that the “offset vector” in the definition of a linear set is required to be nonnegative.

Note that by appropriate integer arithmetic, a partial function $f: \mathbb{N}^k \dashrightarrow \mathbb{N}^l$ is affine if and only if there exist kl integers $n_{1,1}, \dots, n_{k,l} \in \mathbb{Z}$ and $2l+k$ nonnegative integers $b_1, \dots, b_l, c_1, \dots, c_k, d_1, \dots, d_l \in \mathbb{N}$ such that, if $\mathbf{y} = f(\mathbf{x})$, then for each $j \in \{1, \dots, l\}$,

$y(j) = b_j + \frac{1}{d_j} \sum_{i=1}^k n_{i,j}(\mathbf{x}(i) - c_i)$, and for each $i \in \{1, \dots, k\}$, $\mathbf{x}(i) - c_i \geq 0$. Each d_j may be taken to be the least common multiple of the denominators of the rational coefficients in the original definition. We will employ this latter definition when convenient.

2.1 Chemical reaction networks

If Λ is a finite set (in this paper, of chemical species), we write $\mathbb{N}^\Lambda$ to denote the set of functions $f: \Lambda \rightarrow \mathbb{N}$. Equivalently, we view an element $\mathbf{c} \in \mathbb{N}^\Lambda$ as a vector of $|\Lambda|$ nonnegative integers, with each coordinate “labeled” by an element of Λ . Given $X \in \Lambda$ and $\mathbf{c} \in \mathbb{N}^\Lambda$, we refer to $\mathbf{c}(X)$ as the *count of X in c*. We write $\mathbf{c} \leq \mathbf{c}'$ to denote that $\mathbf{c}(X) \leq \mathbf{c}'(X)$ for

all $X \in \Lambda$. Given $\mathbf{c}, \mathbf{c}' \in \mathbb{N}^\Lambda$, we define the vector component-wise operations of addition $\mathbf{c} + \mathbf{c}'$, subtraction $\mathbf{c} - \mathbf{c}'$, and scalar multiplication $n\mathbf{c}$ for $n \in \mathbb{N}$. If $\Delta \subset \Lambda$, we view a vector $\mathbf{c} \in \mathbb{N}^\Delta$ equivalently as a vector $\mathbf{c} \in \mathbb{N}^\Lambda$ by assuming $\mathbf{c}(X) = 0$ for all $X \in \Lambda \setminus \Delta$.

Given a finite set of chemical species Λ , a *reaction* over Λ is a triple $\alpha = \langle \mathbf{r}, \mathbf{p}, k \rangle \in \mathbb{N}^\Lambda \times \mathbb{N}^\Lambda \times \mathbb{R}^+$, specifying the stoichiometry of the reactants and products, respectively, and the *rate constant* k . If not specified, assume that $k = 1$ (this is the case for all reactions in this paper), so that the reaction $\alpha = \langle \mathbf{r}, \mathbf{p}, 1 \rangle$ is also represented by the pair $\langle \mathbf{r}, \mathbf{p} \rangle$. For instance, given $\Lambda = \{A, B, C\}$, the reaction $A + 2B \rightarrow A + 3C$ is the pair $\langle (1, 2, 0), (1, 0, 3) \rangle$. A (*finite*) *chemical reaction network (CRN)* is a pair $C = (\Lambda, R)$, where Λ is a finite set of chemical species, and R is a finite set of reactions over Λ . A *configuration* of a CRN $C = (\Lambda, R)$ is a vector $\mathbf{c} \in \mathbb{N}^\Lambda$. We also write $\#_{\mathbf{c}}X$ to denote $\mathbf{c}(X)$, the *count* of species X in configuration $\mathbf{c}$, or simply $\#X$ when $\mathbf{c}$ is clear from context.

Given a configuration $\mathbf{c}$ and reaction $\alpha = \langle \mathbf{r}, \mathbf{p} \rangle$, we say that α is *applicable* to $\mathbf{c}$ if $\mathbf{r} \leq \mathbf{c}$ (i.e., $\mathbf{c}$ contains enough of each of the reactants for the reaction to occur). If α is applicable to $\mathbf{c}$, then write $\alpha(\mathbf{c})$ to denote the configuration $\mathbf{c} + \mathbf{p} - \mathbf{r}$ (i.e., the configuration that results from applying reaction α to $\mathbf{c}$). If $\mathbf{c}' = \alpha(\mathbf{c})$ for some reaction $\alpha \in R$, we write $\mathbf{c} \rightarrow_C \mathbf{c}'$, or merely $\mathbf{c} \rightarrow \mathbf{c}'$ when C is clear from context. An *execution* (a.k.a., *execution sequence*) ε is a finite or infinite sequence of one or more configurations $\varepsilon = (\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \dots)$ such that, for all $i \in \{1, \dots, |\varepsilon| - 1\}$, $\mathbf{c}_{i-1} \rightarrow \mathbf{c}_i$. If a finite execution sequence starts with $\mathbf{c}$ and ends with $\mathbf{c}'$, we write $\mathbf{c} \xrightarrow{*}_C \mathbf{c}'$, or merely $\mathbf{c} \xrightarrow{*} \mathbf{c}'$ when the CRN C is clear from context. In this case, we say that $\mathbf{c}'$ is *reachable* from $\mathbf{c}$.

Turing machines, for example, have different semantic interpretations depending on the computational task under study (deciding a language, computing a function, etc.). Similarly, in this paper we use CRNs to decide subsets of $\mathbb{N}^k$ and to compute functions $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$. In the next two subsections we define two semantic interpretations of CRNs that correspond to these two tasks.

2.2 Stable decidability of predicates

We now review the definition of stable decidability of predicates introduced by Angluin, Aspnes, and Eisenstat [2].³ Intuitively, some species “vote” for a true/false answer and the system stabilizes to an output when a consensus is reached and it can no longer change its mind. The determinism of the system is captured in that it is impossible to stabilize to an incorrect answer, and the correct stable output is always reachable.

A *chemical reaction decider (CRD)* is a tuple $D = (\Lambda, R, \Sigma, Y, \varphi, \sigma)$, where (Λ, R) is a CRN, $\Sigma \subseteq \Lambda$ is the *set of input species*, $Y \subseteq \Lambda$ is the set of *voters*⁴, $\varphi: Y \rightarrow \{0, 1\}$ is the (*Boolean*) *output function*, and $\sigma \in \mathbb{N}^{\Lambda \setminus \Sigma}$ is the *initial context*. An input to D will be a vector $\mathbf{i}_0 \in \mathbb{N}^\Sigma$ (equivalently, $\mathbf{i}_0 \in \mathbb{N}^k$ if we write $\Sigma = \{X_1, \dots, X_k\}$ and assign X_i to represent the i 'th

³Those authors use the term “stably compute”, but we reserve the term “compute” to apply to the computation of functions $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$.

⁴The definitions of [2] assume that $Y = \Lambda$ (i.e., every species votes). However, it is not hard to show that we may equivalently assume there are only two voting species, F and T , so that $\#F > 0$ and $\#T = 0$ means that the CRD is answering “false”, and $\#F = 0$ and $\#T > 0$ means that the CRD is answering “true.” This convention will be more convenient in this paper.

coordinate). Thus a CRD together with an input vector defines an initial configuration $\mathbf{i}$ defined by $\mathbf{i}(X) = \mathbf{i}_0(X)$ if $X \in \Sigma$, and $\mathbf{i}(X) = \sigma(X)$ otherwise. We say that such a configuration is a *valid initial configuration*, i.e., $\mathbf{i} \vdash (\Lambda \setminus \Sigma) = \sigma$. If we are discussing a CRN understood from context to have a certain initial configuration $\mathbf{i}$, we write $\#_0 X$ to denote $\mathbf{i}(X)$.

We extend φ to a partial function $\Phi : \mathbb{N}^\Lambda \rightarrow \{0, 1\}$ as follows. $\Phi(\mathbf{c})$ is undefined if either $\mathbf{c}(X) = 0$ for all $X \in Y$, or if there exist $X_0, X_1 \in Y$ such that $\mathbf{c}(X_0) > 0$, $\mathbf{c}(X_1) > 0$, $\varphi(X_0) = 0$ and $\varphi(X_1) = 1$. Otherwise, there exists $b \in \{0, 1\}$ such that $(\forall X \in Y)(\mathbf{c}(X) > 0 \Rightarrow \varphi(X) = b)$; in this case, the *output* $\Phi(\mathbf{c})$ of configuration $\mathbf{c}$ is b .

A configuration $\mathbf{c}$ is *output stable* if $\Phi(\mathbf{c})$ is defined and, for all $\mathbf{c}'$ such that $\mathbf{c} \rightarrow^* \mathbf{c}'$, $\Phi(\mathbf{c}') = \Phi(\mathbf{c})$.⁵ We say a CRD D *stably decides* the predicate $\psi : \mathbb{N}^\Sigma \rightarrow \{0, 1\}$ if, for any valid initial configuration $\mathbf{i} \in \mathbb{N}^\Lambda$ with $\mathbf{i} \vdash \Sigma = \mathbf{i}_0$, for all configurations $\mathbf{c} \in \mathbb{N}^\Lambda$, $\mathbf{i} \rightarrow^* \mathbf{c}$ implies $\mathbf{c} \rightarrow^* \mathbf{c}'$ such that $\mathbf{c}'$ is output stable and $\Phi(\mathbf{c}') = \psi(\mathbf{i}_0)$. Note that this condition implies that no incorrect output stable configuration is reachable from $\mathbf{i}$. We say that D *stably decides* a set $A \subseteq \mathbb{N}^k$ if it stably decides its indicator function.

The following theorem is due to Angluin, Aspnes, and Eisenstat [2]:

Theorem 2.1 ([2])—A set $A \subseteq \mathbb{N}^k$ is stably decidable by a CRD if and only if it is semilinear.

The model they use is defined in a slightly different way. They study *population protocols*, a distributed computing model in which a fixed-size set of agents, each having a state from a finite set, undergo successive pairwise interactions, the two agents updating their states upon interacting. This is equivalent to chemical reaction networks in which all reactions have exactly two reactants and two products. However, the result carries over to our more general model, as we now explain.

The reverse direction, that every semilinear predicate is decided by some CRD, follows directly from the result on population protocols, since population protocols are a subset of the set of all CRNs. The forward direction of Theorem 2.1, that every stably decidable set is semilinear, holds even if stable decidability is defined with respect to *any* relation $\rightarrow^*$ on $\mathbb{N}^k$ that is reflexive, transitive, and “respects addition”, i.e., $[(\forall \mathbf{c}_1, \mathbf{c}_2, \mathbf{x} \in \mathbb{N}^k) (\mathbf{c}_1 \rightarrow^* \mathbf{c}_2) \Rightarrow (\mathbf{c}_1 + \mathbf{x} \rightarrow^* \mathbf{c}_2 + \mathbf{x})]$. These properties can easily be shown to hold for the CRN reachability relation. The third property, in particular, means that if some molecules $\mathbf{c}_1$ can react to form molecules $\mathbf{c}_2$, then it is possible for them to react in the presence of some extra molecules $\mathbf{x}$, such that no molecules from $\mathbf{x}$ react at all.

⁵Note that reactions may be applicable in an output stable state $\mathbf{c}$. The same holds for our (very similar) definition of output stable states of CRNs that compute functions instead of predicates, defined in Section 2.3. The definition simply requires that no sequence of these reactions can either 1) produce a molecule that votes contrary to $\Phi(\mathbf{c})$, or 2) consume all molecules voting $\Phi(\mathbf{c})$. Our systematic construction in Lemma 4.4 obeys the stronger constraint that every output-stable state is “static”: no reactions are applicable to it. Thus requiring output stable states to be static does not alter the class of functions stably computable by CRNs. However, the time to convergence proven in Theorem 5.2 is sensitive to this choice, since our construction for Theorem 5.2 reaches an output stable state in expected time $O(\text{polylog } n)$, but reactions continue to occur for expected time $\Omega(\text{poly } n)$.

2.3 Stable computation of functions

Aspnes and Ruppert [4] describe an extension from Boolean predicates to functions, by generalizing the output function φ to a larger range $\varphi: Y \rightarrow \{0, \dots, l\}$. Equivalently, one can consider multiple voting species $V^0, \dots, V^l$; if the CRN converges to only V^j votes, then then output is j . However, this output encoding handles only bounded-range functions: the size of Y must be at least the range of the function. By contrast our results concern functions with unbounded range.

We now define a notion of stable computation of *functions* in which the output and input are encoded identically — in molecular counts of certain species. Intuitively, the inputs to the function are the initial counts of input species $X_1, \dots, X_k$, and the outputs are the counts of output species $Y_1, \dots, Y_l$. The system stabilizes to an output when the counts of the output species can no longer change. Again determinism is captured in that it is impossible to stabilize to an incorrect answer and the correct stable output is always reachable.

Let $k, l \in \mathbb{Z}^+$. A *chemical reaction computer (CRC)* is a tuple $C = (\Lambda, R, \Sigma, \Gamma, \sigma)$, where (Λ, R) is a CRN, $\Sigma \subset \Lambda$ is the *set of input species*, $\Gamma \subset \Lambda$ is the *set of output species*, such that $\Sigma \cap \Gamma = \emptyset$, $|\Sigma| = k$, $|\Gamma| = l$, and $\sigma \in \mathbb{N}^{\Lambda \setminus \Sigma}$ is the *initial context*. Write $\Sigma = \{X_1, X_2, \dots, X_k\}$ and $\Gamma = \{Y_1, Y_2, \dots, Y_l\}$. We say that a configuration $\mathbf{c}$ is *output count stable* if, for every $\mathbf{c}'$ such that $\mathbf{c} \rightarrow^* \mathbf{c}'$ and every $Y_i \in \Gamma$, $\mathbf{c}(Y_i) = \mathbf{c}'(Y_i)$ (i.e., the counts of species in Γ will never change if $\mathbf{c}$ is reached). As with CRD's, we require initial configurations $\mathbf{i}$ of C with input $\mathbf{i}_0 \in \mathbb{N}^\Sigma$ to obey $\mathbf{i}(X) = \mathbf{i}_0(X)$ if $X \in \Sigma$ and $\mathbf{i}(X) = \sigma(X)$ otherwise, calling them *valid initial configurations*. We say that C *stably computes* a function $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ if for any valid initial configuration $\mathbf{i} \in \mathbb{N}^\Lambda$, $\mathbf{i} \rightarrow^* \mathbf{c}$ implies $\mathbf{c} \rightarrow^* \mathbf{c}'$ such that $\mathbf{c}'$ is an output count stable configuration with $f(\mathbf{i}(X_1), \mathbf{i}(X_2), \dots, \mathbf{i}(X_k)) = (\mathbf{c}'(Y_1), \mathbf{c}'(Y_2), \dots, \mathbf{c}'(Y_l))$. Note that this condition implies that no incorrect output stable configuration is reachable from $\mathbf{i}$.

As an example of a formally defined CRC consider the function $f(x) = \lfloor x/2 \rfloor$ shown in Fig. 1(a). This function is stably computed by the CRC $(\Lambda, R, \Sigma, \Gamma, \sigma)$ where (Λ, R) is the CRN consisting of a single reaction $2X \rightarrow Y$, $\Sigma = \{X\}$ is the set of input species, $\Gamma = \{Y\}$ is the set of output species, and the initial context σ is zero for all species in $\Lambda \setminus \Sigma$. In Fig. 2(b) the initial context $\sigma(F) = 1$, and is zero for all other species in $\Lambda \setminus \Sigma$. In Fig. 1(a) there is at most one reaction that can happen in any reachable configuration. In contrast, different reactions may occur next in Fig. 1(b) and (c). However, from any reachable state, we can reach the output count stable configuration with the correct amount of Y , satisfying our definition of stable computation.

In Sections 4–5 we will describe systematic (but much more complex) constructions for these and all functions with semilinear graphs.

2.4 Fair execution sequences

Note that by defining deterministic computation in terms of certain states being reachable and others not, we cannot guarantee the system will get to the correct output *for any possible execution sequence*. For example suppose an adversary controls the execution sequence.

Then $\{X \rightarrow 2Y, A \rightarrow B, B \rightarrow A\}$ will not reach the intended output state $y = 2x$ if the adversary simply does not let the first reaction occur, always preferring the second or third.

Intuitively, in a real chemical mixture, the reactions are chosen randomly and not adversarially, and the CRN will get to the correct output. In this section we follow Angluin, Aspnes, and Eisenstat [2] and define a combinatorial condition called fairness on execution sequences that captures what is minimally required of the execution sequence to be guaranteed that a stably deciding/computing CRD/CRC will reach the output stable state. In the next section we consider the kinetic model, which ascribes probabilities to execution sequences. The kinetic model also defines the time of reactions, allowing us to study the computational complexity of the CRN computation. Note that in the kinetic model, if the reachable configuration space is bounded for any start configuration (i.e. if from any starting configuration there are finitely many configurations reachable) then any observed execution sequence will be fair with probability 1. This will be the case for our constructions in Sections 4 and 5.

Let $\Delta \subseteq \Lambda$. We say that $\mathbf{p} \in \mathbb{N}^\Delta$ is a *partial configuration* (with respect to Δ). We write $\mathbf{p} = \mathbf{c} \upharpoonright \Delta$ for any configuration $\mathbf{c}$ such that $\mathbf{c}(X) = \mathbf{p}(X)$ for all $X \in \Delta$, and we say that $\mathbf{p}$ is the *restriction of $\mathbf{c}$ to Δ* . Say that a partial configuration $\mathbf{p}$ with respect to Δ is *reachable* from configuration $\mathbf{c}'$ if there is a configuration $\mathbf{c}$ reachable from $\mathbf{c}'$ and $\mathbf{p} = \mathbf{c} \upharpoonright \Delta$. In this case, we write $\mathbf{c}' \rightarrow^* \mathbf{p}$.

An infinite execution $\varepsilon = (\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \dots)$ is *fair* if, for all partial configurations $\mathbf{p}$, if $\mathbf{p}$ is infinitely often reachable then it is infinitely often reached.⁶ In other words, no reachable partial configuration is “starved”.⁷ This definition, applied to finite executions, deems all of them fair vacuously. We wish to distinguish between finite executions that can be extended by applying another reaction and those that cannot. Say that a configuration is *terminal* if no reaction is applicable to it. We say that a finite execution is *fair* if and only if it ends in a terminal configuration. For any species $A \in \Lambda$, we write $\#_\infty A$ to denote the eventual convergent count of A if $\#A$ is guaranteed to stabilize on any fair execution sequence; otherwise, $\#_\infty A$ is undefined.

The next lemma characterizes stable computation of functions by CRCs in terms of fair execution sequences, showing that the counts of output species will converge to the correct output values on any fair execution sequence. An analogous lemma holds for CRDs.

Lemma 2.2—A CRC stably computes a function $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ if and only if for every valid initial configuration $\mathbf{i} \in \mathbb{N}^\Lambda$, every fair execution $E = (\mathbf{i}, \mathbf{c}_1, \mathbf{c}_2, \dots)$ contains an output count stable configuration $\mathbf{c}$ such that $f(\mathbf{i}(X_1), \mathbf{i}(X_2), \dots, \mathbf{i}(X_k)) = (\mathbf{c}(Y_1), \mathbf{c}(Y_2), \dots, \mathbf{c}(Y_l))$.

⁶i.e. $(\forall \Delta \subseteq \Lambda)(\forall \mathbf{p} \in \mathbb{N}^\Delta)[((\exists^\infty i \in \mathbb{N}) \mathbf{c}_i \rightarrow^* \mathbf{p}) \Rightarrow ((\exists^\infty j \in \mathbb{N}) \mathbf{p} = \mathbf{c}_j \upharpoonright \Delta)]$.

⁷This definition of fairness is stricter than that used in [2], which used only full configurations rather than partial configurations. We choose this definition to prevent intuitively unfair executions from vacuously satisfying the definition of “fair” simply because of some species whose count is monotonically increasing with time (preventing any configuration from being infinitely often reachable). Such a definition is unnecessary in [2] because population protocols by definition have a finite state space, since they enforce that every reaction has precisely two reactants and two products.

Proof: The “if” direction follows because every finite execution sequence can be extended to be fair, and thus an output count stable configuration with the correct output is always reachable. The “only if” direction is shown as follows. We know that from any reachable configuration $\mathbf{c}$, some correct output stable configuration $\mathbf{c}'$ is reachable (but possibly different $\mathbf{c}'$ for different $\mathbf{c}$). We will argue that in any infinite fair execution sequence there is some partial configuration that is reachable infinitely often, and that any state with this partial configuration is the correct stable output state. Consider an infinite fair execution sequence $\mathbf{c}_1, \mathbf{c}_2, \dots$, and the corresponding reachable correct output stable configurations $\mathbf{c}'_1, \mathbf{c}'_2, \dots$. As in Lemma 11 of [2], there is some integer $k \geq 1$ such that a configuration is output count stable if and only if it is output count stable when each coordinate that is larger than k is set to exactly k (k -truncation). The infinite sequence $\mathbf{c}'_1, \mathbf{c}'_2, \dots$ must have an infinite subsequence sharing the same k -truncation. Let $\mathbf{p}$ be the partial configuration consisting of the correct output and all the coordinates less than k in the shared truncation. This partial configuration is reachable infinitely often, and no matter what the counts of the other species outside of $\mathbf{p}$ are, the resulting configuration is output count stable.

2.5 Kinetic model

The following model of stochastic chemical kinetics is widely used in quantitative biology and other fields dealing with chemical reactions between species present in small counts [13]. It ascribes probabilities to execution sequences, and also defines the time of reactions, allowing us to study the computational complexity of the CRN computation in Sections 4 and 5.

In this paper, the rate constants of all reactions are 1, and we define the kinetic model with this assumption. A reaction is *unimolecular* if it has one reactant and *bimolecular* if it has two reactants. We use no higher-order reactions in this paper when using the kinetic model.

The kinetics of a CRN is described by a continuous-time Markov process as follows. Given a fixed volume v and current configuration $\mathbf{c}$, the *propensity* of a unimolecular reaction $\alpha : X \rightarrow \dots$ in configuration $\mathbf{c}$ is $\rho(\mathbf{c}, \alpha) = \#_{\mathbf{c}} X$. The propensity of a bimolecular reaction $\alpha : X + Y \rightarrow \dots$, where $X \neq Y$, is $\rho(\mathbf{c}, \alpha) = \frac{\#_{\mathbf{c}} X \#_{\mathbf{c}} Y}{v}$. The propensity of a bimolecular reaction $\alpha : X + X \rightarrow \dots$ is $\rho(\mathbf{c}, \alpha) = \frac{1}{2} \frac{\#_{\mathbf{c}} X (\#_{\mathbf{c}} X - 1)}{v}$. The propensity function determines the evolution of the system as follows. The time until the next reaction occurs is an exponential random variable with rate $\rho(\mathbf{c}) = \sum_{\alpha \in R} \rho(\mathbf{c}, \alpha)$ (note that $\rho(\mathbf{c}) = 0$ if no reactions are applicable to $\mathbf{c}$).

The probability that next reaction will be a particular α_{next} is $\frac{\rho(\mathbf{c}, \alpha_{\text{next}})}{\rho(\mathbf{c})}$.

The kinetic model is based on the physical assumption of well-mixedness that is valid in a dilute solution. Thus, we assume the *finite density constraint*, which stipulates that a volume required to execute a CRN must be proportional to the maximum molecular count obtained during execution [24]. In other words, the total concentration (molecular count per volume) is bounded. This realistically constrains the speed of the computation achievable by CRNs. Note, however, that it is problematic to define the kinetic model for CRNs in which the

reachable configuration space is unbounded for some start configurations, because this means that arbitrarily large molecular counts are reachable.⁸ We apply the kinetic model only to CRNs with configuration spaces that are bounded for each start configuration.

We now prove two lemmas about the complexity of certain common sequences of reactions. Besides providing simple examples of the kinetic model, they capture patterns that will be used throughout Section 4 and 5. These lemmas are implicit or explicit in many earlier papers on stochastic CRNs.

Lemma 2.3—Let $\{A_1, \dots, A_m\}$ be a set of species, such that the count of each is $O(n)$. Then the expected time for i unimolecular reactions $A_i \rightarrow \dots$, in which none of the A_i appear as products, to consume all A_i 's is $O(\log n)$.

Proof: In any configuration $\mathbf{c}$, the propensity of the i th reaction is $\#_{\mathbf{c}} A_i$. Let $k = \sum_i \#_{\mathbf{c}} A_i$. The time until next reaction is an exponential random variable with propensity k . Thus the expected time until the next reaction occurs is $1/k$. Every time one of the reactions occurs, one of the A_i 's is consumed, and so k decreases by 1. Thus, by linearity of expectation, the expected time to consume all the A_i molecules is $\sum_{k=1}^{O(n)} 1/k = O(\log n)$.

Lemma 2.4—Let L be a species with count 1, and A a species of count n . Then, if the volume is $v = O(n)$, the expected time for reaction $L + A \rightarrow L + B$ to convert all A 's to B 's is $O(n \log n)$.

Proof: When exactly k molecules of species A remain, the propensity of the reaction is k/v . Thus the expected time until the next reaction is v/k . Therefore by linearity of expectation, the expected time for L to react with every A is $\sum_{k=1}^n \frac{v}{k} = v \cdot O(\log(n)) = O(n \log n)$.

3 Exactly the semilinear functions can be deterministically computed

In this section we use Theorem 2.1 to show that only “simple” functions can be stably computed by CRCs. This is done by showing how to reduce the computation of a function by a CRC to the decidability of its graph by a CRD, and vice versa. In this section we do not concern ourselves with kinetics. Thus the volume is left unspecified, and we consider the combinatorial-only condition of fairness on execution sequences for our positive result (Lemma 3.2) and direct reachability arguments for the negative result (Lemma 3.1).

The next lemma shows that every function computable by a chemical reaction network is semilinear by reducing stably deciding a set that is the graph of a function to stably computing that function. It turns out that the reduction technique of introducing “production” and “consumption” indicator species will be a general technique, used repeatedly in this paper.

⁸One possibility is to have a “dynamically” growing volume as in [24].

Lemma 3.1

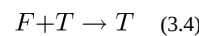
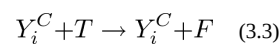
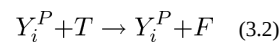
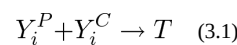
Every function stably computable by a CRC is semilinear.

Proof—Suppose there is a CRC C stably computing f . We will construct a CRD D that stably decides the graph of f . By Theorem 2.1, this implies that the graph of f is semilinear. Intuitively, the difficulty lies in checking whether the amount of the outputs Y_i produced by C matches the value given to the decider D as input. What makes this non-trivial is that D does not know whether C has finished computing, and thus must compare Y_i while Y_i is potentially being changed by C . In particular, D cannot consume Y_i or that could interfere with the operation of C .

Let $C = (\Lambda, R, \Sigma, \Gamma, \sigma)$ be the CRC that stably computes $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$, with input species $\Sigma = \{X_1, \dots, X_k\}$ and output species $\Gamma = \{Y_1, \dots, Y_l\}$. We will modify C to obtain the following CRD $D = (\Lambda', R', \Sigma', \Gamma', \varphi', \sigma')$. Let $Y^C = \{Y_1^C, \dots, Y_l^C\}$ and $Y^P = \{Y_1^P, \dots, Y_l^P\}$, where each $Y_i^C, Y_i^P \notin \Lambda$ are new species. Intuitively, $\#Y_i^P$ represents the number of Y_i 's produced by C and $\#Y_i^C$ the number of Y_i 's consumed by C . The goal is for D to stably decide the predicate $f(\#_0 X_1, \dots, \#_0 X_k) = (\#_0 Y_1^C, \dots, \#_0 Y_l^C)$. In other words, the initial configuration of D will be the same as that of C except for some copies of Y_i^C , equal to the purported output of f to be tested by D .

Let $\Lambda' = \Lambda \cup Y^C \cup Y^P \cup \{F, T\}$. Let $\Sigma' = \Sigma \cup Y^C$. Let $\Gamma' = \{F, T\}$, with $\varphi(F) = 0$ and $\varphi(T) = 1$. Let $\sigma'(T) = 1$ and $\sigma'(S) = 0$ for all $S \in \Lambda \setminus (\Sigma' \cup \{T\})$. We will modify R to obtain R' as follows. For each reaction α that consumes a net number n of Y_i molecules, append n products Y_i^C to α . For each reaction α that produces a net number n of Y_i molecules, append n products Y_i^P to α . For example, the reaction $A + 2B + Y_1 + 3Y_3 \rightarrow Z + 3Y_1 + 2Y_3$ becomes $A + 2B + Y_1 + 3Y_3 \rightarrow Z + 3Y_1 + 2Y_3 + 2Y_1^P + Y_3^C$.

Then add the following additional reactions to R' , for each $i \in \{1, \dots, l\}$,



Observe that if $f(\#_0 X_1, \dots, \#_0 X_k) = (\#_0 Y_1^C, \dots, \#_0 Y_l^C)$, then from any reachable configuration we can reach a configuration without any Y_i^P or Y_i^C for all i , and such that no more of either kind can be produced. (The CRC stabilizes and all of Y_i^P and Y_i^C is consumed by reaction 3.1.) In this configuration we must have $\#T > 0$ because the last

instance of reaction 3.1 produced it (or if no output was ever produced, T comes from the initial context σ), and T can no longer be consumed in reactions 3.2–3.3. Thus, since all of F can be consumed in reaction 3.4, a configuration with $\#T > 0$ and $\#F = 0$ is always reachable, and this configuration is output stable.

Now suppose $f(\#_0 X_1, \dots, \#_0 X_k) \neq (\#_0 Y_1^C, \dots, \#_0 Y_l^C)$ for some output coordinate $i^* \in \{1, \dots, l\}$. This means that from any reachable configuration we can reach a configuration with either $\#Y_{i^*}^P > 0$ or $\#Y_{i^*}^C > 0$ but not both, and such that for all i , no more of Y_i^P and Y_i^C can be produced. (This happens when the CRC stabilizes and reaction 3.1 consumes the smaller of $Y_{i^*}^P$ or $Y_{i^*}^C$.) From this configuration, we can reach a configuration with $\#F > 0$ and $\#T = 0$ through reactions 3.2–3.3. This is an output stable configuration since reactions 3.2–3.4 require T .

The next lemma shows the converse of Lemma 3.1. Intuitively, it uses a random search of the output space to look for the correct answer to the function and uses a predicate decider to check whether the correct solution has been found.

Lemma 3.2

Every semilinear function is stably computable by a CRC.

Proof—Let $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ be a semilinear function, and let

$$G = \{(\mathbf{x}, \mathbf{y}) \in \mathbb{N}^k \times \mathbb{N}^l \mid f(\mathbf{x}) = \mathbf{y}\}$$

denote the graph of f . We then consider the set

$$\hat{G} = \{(\mathbf{x}, \mathbf{y}_P, \mathbf{y}_C) \in \mathbb{N}^k \times \mathbb{N}^l \times \mathbb{N}^l \mid f(\mathbf{x}) = \mathbf{y}_P - \mathbf{y}_C\}.$$

Intuitively, $\hat{G}$ defines the same function as G , but with each output variable expressed as the difference between two other variables. Note that $\hat{G}$ is not the graph of a function since for each $\mathbf{y} \in \mathbb{N}^l$ there are many pairs $(\mathbf{y}_P, \mathbf{y}_C)$ such that $\mathbf{y}_P - \mathbf{y}_C = \mathbf{y}$. However, we only care that $\hat{G}$ is a semilinear set so long as G is a semilinear set, by Lemma 3.3, proven below.

Then by Theorem 2.1, $\hat{G}$ is stably decidable by a CRD $D = (\Lambda, R, \Sigma, Y, \phi, \sigma)$, where

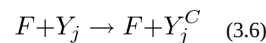
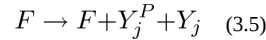
$$\Sigma = \{X_1, \dots, X_k, Y_1^P, \dots, Y_l^P, Y_1^C, \dots, Y_l^C\},$$

and we assume that Y contains only species T and F such that for any output-stable configuration of D , exactly one of $\#T$ or $\#F$ is positive to indicate a true or false answer, respectively.

Define the CRC $C = (\Lambda', R', \Sigma', \Gamma', \sigma')$ as follows. Let $\Sigma' = \{X_1, \dots, X_k\}$. Let $\Gamma' = \{Y_1, \dots, Y_l\}$. Let $\Lambda' = \Lambda \cup \Gamma'$. Let $\sigma'(S) = \sigma(S)$ for all $S \in \Lambda \setminus \Sigma$, and let $\sigma'(S) = 0$ for all $S \in \Lambda' \setminus (\Lambda \setminus$

Σ). Intuitively, we will have F change the value of $\mathbf{y}$ (by producing either Y_j^P or Y_j^C molecules), since F 's presence indicates that D has not yet decided that the predicate is satisfied. It essentially searches for new values of $\mathbf{y}$ that do satisfy the predicate. This indirect way of representing the value $\mathbf{y}$ is useful because $\mathbf{y}_P$ and $\mathbf{y}_C$ can both be increased monotonically to change $\mathbf{y}$ in either direction. If D had Y_j as a species directly, and if we wanted to test a lower value of $\mathbf{y}_j$, then this would require consuming a copy of Y_j , but this may not be possible if D has already consumed all of them.

Let R' be R plus the following reactions for each $j \in \{1, \dots, l\}$:



It is clear that reactions (3.5) and (3.6) enforce that at any time, $\#Y_j$ is equal to the total number of Y_j^P 's produced by reaction (3.5) minus the total number of Y_j^C 's produced by reaction (3.6) (although some of each of Y_j^P or Y_j^C may have been produced or consumed by other reactions in R).

Suppose that $f(\mathbf{x}) \neq (\#Y_1, \dots, \#Y_l)$. Then if there are no F molecules present, the counts of Y_j^P and Y_j^C are not changed by reactions (3.5) and (3.6). Therefore only reactions in R proceed, and by the correctness of D , eventually an F molecule is produced (since eventually D must reach an output-stable configuration answering “false”, although F may appear before D reaches an output-stable configuration, if some T are still present). Once F is present, by the fairness condition (choosing $\Delta = \{Y_1, \dots, Y_l\}$), eventually the value of $(\#Y_1, \dots, \#Y_l)$ will change by reaction (3.5) or (3.6). In fact, every value of $(\#Y_1, \dots, \#Y_l)$ is possible to explore by the fairness condition.

Suppose then that $f(\mathbf{x}) = (\#Y_1, \dots, \#Y_l)$. Perhaps F is present because the reactions in R have not yet reached an output-stable “true” configuration. Then perhaps the value of $(\#Y_1, \dots, \#Y_l)$ will change so that $f(\mathbf{x}) \neq (\#Y_1, \dots, \#Y_l)$. But by the fairness condition, a correct value of $(\#Y_1, \dots, \#Y_l)$ must be present infinitely many times, so again by the fairness condition, since from such a configuration it is possible to eliminate all F molecules before producing Y_j^P or Y_j^C molecules, this must eventually happen. When all F molecules are gone while $f(\mathbf{x}) = (\#Y_1, \dots, \#Y_l)$ and D is in an output-stable configuration (thus no F can ever again be produced), then it is no longer possible to change the value of $(\#Y_1, \dots, \#Y_l)$, whence C has reached a count-stable configuration with the correct answer. Therefore C stably computes f .

Note that the total molecular count (hence the required volume) of the CRC in Lemma 3.2 is unbounded. In Section 4 we discuss an alternative construction that avoids this problem.

Lemma 3.3

Let $k, l \in \mathbb{Z}^+$, and suppose $G \subseteq \mathbb{N}^k \times \mathbb{N}^l$ is semilinear. Define

$$\hat{G} = \left\{ (\mathbf{x}, \mathbf{y}_P, \mathbf{y}_C) \in \mathbb{N}^k \times \mathbb{N}^l \times \mathbb{N}^l \mid (\mathbf{x}, \mathbf{y}_P - \mathbf{y}_C) \in G \right\}.$$

Then $\hat{G}$ is semilinear.

Proof—Let $G_1, \dots, G_t$ be linear sets such that $G = \bigcup_{i=1}^t G_i$. For each $i \in \{1, \dots, t\}$, define

$$\hat{G}_i = \left\{ (\mathbf{x}, \mathbf{y}_P, \mathbf{y}_C) \in \mathbb{N}^k \times \mathbb{N}^l \times \mathbb{N}^l \mid (\mathbf{x}, \mathbf{y}_P - \mathbf{y}_C) \in G_i \right\}.$$

It suffices to show that each $\hat{G}_i$ is linear since $\hat{G} = \bigcup_{i=1}^t \hat{G}_i$. Let $i \in \{1, \dots, t\}$ and let $\mathbf{b}, \mathbf{u}_1, \dots, \mathbf{u}_r \in \mathbb{N}^k \times \mathbb{N}^l$ be such that

$$G_i = \left\{ \mathbf{b} + \sum_{j=1}^r n_j \mathbf{u}_j \mid n_j \in \mathbb{N} \right\}.$$

Define the vectors $\mathbf{v}_1, \dots, \mathbf{v}_r \in \mathbb{N}^k \times \mathbb{N}^l \times \mathbb{N}^l$ as $\mathbf{v}_j = (\mathbf{u}_j, \mathbf{0}^l)$. Here, $\mathbf{0}^l$ denotes the vector in $\mathbb{N}^l$ consisting of all zeros. In other words, let $\mathbf{v}_j$ be $\mathbf{u}_j$ on its first $k + l$ coordinates and 0 on its last l coordinates. Similarly define $\mathbf{b}' = (\mathbf{b}, \mathbf{0}^l)$.

Also, for each $j \in \{1, \dots, l\}$ define $\mathbf{v}_{r+j} = (0^k, 0^{l-j}, 0^{j-1}, 1, 0^{l-j})$. (i.e., a single 1 in the position corresponding to the j th output coordinate, one for $\mathbf{y}_P$ and one for $\mathbf{y}_C$). Without the vectors $\mathbf{v}_{r+j}$, the set of points defined by $\mathbf{b}', \mathbf{v}_1, \dots, \mathbf{v}_r$ would be simply G_i with l 0's appended to the end of each vector. By adding the vectors $\mathbf{v}_{r+j}$, for each $(\mathbf{x}, \mathbf{y}) \in G_i$ and each $\mathbf{y}_P, \mathbf{y}_C \in \mathbb{N}^l$ such that $\mathbf{y} = \mathbf{y}_P - \mathbf{y}_C$, we have that $(\mathbf{x}, \mathbf{y}_P, \mathbf{y}_C) = \mathbf{b}' + \sum_{j=1}^{r+l} n_j \mathbf{v}_j$ for some $n_1, \dots, n_{r+l} \in \mathbb{N}$; in particular, for $n_1, \dots, n_r$ chosen such that $(\mathbf{x}, \mathbf{y}) = \mathbf{b} + \sum_{j=1}^r n_j \mathbf{u}_j$ and $n_{r+j} = \mathbf{y}_C(j)$ for each $j \in \{1, \dots, l\}$.

Thus $\hat{G}_i = \left\{ \mathbf{b}' + \sum_{j=1}^{r+l} n_j \mathbf{v}_j \mid n_j \in \mathbb{N} \right\}$, whence $\hat{G}_i$ is linear.

Lemmas 3.1 and 3.2 immediately imply the following theorem.

Theorem 3.4

A function $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ is stably computable by a CRC if and only if it is semilinear.

One unsatisfactory aspect of Lemma 3.2 is that we do not reduce the computation of f directly to a CRD deciding the graph G of f , but rather to D deciding a related set $\hat{G}$. It is not clear how to directly reduce to a CRD deciding G since it is not obvious how to modify such a CRD to monotonically produce extra species that could be processed by the CRC computing f . Lemma 3.1, on the other hand, directly uses C as a black-box. Although we know that C , being a chemical reaction computer, is only capable of computing semilinear functions, if we imagine that some external powerful “oracle” controlled the reactions of C

to allow it to stably compute a non-semilinear function, then D would decide that function's graph. Thus Lemma 3.1 is more like the black-box oracle Turing machine reductions employed in computability and complexity theory, which work no matter what mythical device is hypothesized to be responsible for answering the oracle queries.

4 Deterministic computation of semilinear functions in $O(\|x\| \log \|x\|)$ time

Lemma 3.2 describes how a CRC can deterministically compute any semilinear function. However, there are problems with this construction if we attempt to use it to evaluate the speed of semilinear function computation in the kinetic model. First, the configuration space is unbounded for any input since the construction searches over outputs without setting bounds. Thus, more care must be taken to ensure that any infinite execution sequence will be fair with probability 1 in the kinetic model. What is more, since the maximum molecular count is unbounded, it is not clear how to set the volume for the time analysis. Even if we attempt to properly define kinetics, it seems like any reasonable time analysis of the random search process will result in expected time at least exponential in the size of the output.⁹

For our asymptotic time analysis, let the input size $n = \|x\|$ be the number of input molecules. The total molecular count attainable will always be $O(n)$; thus, by the finite density constraint, we assume the volume $v = \Theta(n)$. We now describe a direct construction for computing semilinear functions in $O(n \log n)$ time that does not rely on the search technique explored in the previous section, but rather uses the mathematical structure of the semilinear graph.

For the asymptotic running time analysis, we will repeatedly assume that reactions complete “sequentially”: upstream reactions complete before downstream ones start. Although this is unrealistic, it provides an upper bound on the computation time that is easy to calculate. Note that in proving the correctness of our CRN algorithms we cannot make this assumption, because we must show that the computation is correct no matter in what order the reaction occur.

We use the technique of “running multiple CRNs in parallel” on the same input. To accomplish this it is necessary to split the inputs $X_1, \dots, X_k$ into separate molecules using a reaction $X_i \rightarrow X_i^1 + X_i^2 + \dots + X_i^p$, which will add only $O(\log n)$ to the time complexity by Lemma 2.3, so that each of the p separate parallel CRNs do not interfere with one another. For brevity we omit stating this formally when the technique is used.

We require the following theorem, due to Angluin, Aspnes, Diamadi, Fischer, and René [1], which states that any semilinear predicate can be decided by a CRD in expected time $O(n \log n)$. (This was subsequently reduced to $O(n)$ by Angluin, Aspnes, and Eisenstat [3], but $O(n \log n)$ suffices for our purpose.)

⁹The random walk is biased downward because of the increasing propensities of the reactions consuming Y_i 's.

Theorem 4.1 ([1])

Let $\varphi: \mathbb{N}^k \rightarrow \{0, 1\}$ be a semilinear predicate. Then there is a stable CRD D that decides φ , and the expected time to reach an output-stable state is $O(n \log n)$, where n is the number of input molecules.

The next lemma shows that affine partial functions can be computed in expected time $O(n \log n)$ by a CRC. For its use in proving Theorem 4.4, we require that the output molecules be produced monotonically. Unfortunately, this is impossible for general affine partial functions. For example, consider the function $f(x_1, x_2) = x_1 - x_2$ where the domain of f is $\text{dom } f = \{(x_1, x_2) \mid x_1 \geq x_2\}$. By withholding a single copy of X_2 and letting the CRC stabilize to the output value $\#Y = x_1 - x_2 + 1$, then allowing the extra copy of X_2 to interact, the only way to stabilize to the correct output value $x_1 - x_2$ is to consume a copy of the output species Y . Therefore Lemma 4.2 is stated in terms of an encoding of affine partial functions that allows monotonic production of outputs, encoding the output value $\mathbf{y}(j)$ as the difference between the counts of two monotonically produced species Y_j^P and Y_j^C , using the same technique used in the proofs of Lemmas 3.1 and 3.2.

Let $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ be an affine partial function, where, letting $\mathbf{y} = f(\mathbf{x})$, for all $j \in \{1, \dots, l\}$,

$\mathbf{y}(j) = b_j + \frac{1}{d_j} \sum_{i=1}^k n_{i,j} (\mathbf{x}(i) - c_i)$ for integer $n_{i,j}$ and nonnegative integer b_j, c_i , and d_j . Define $\hat{f}: \mathbb{N}^k \rightarrow \mathbb{N}^l \times \mathbb{N}^l$ as follows. For each $\mathbf{x} \in \text{dom } f$, define $\mathbf{y}_C \in \mathbb{N}^l$ for each $j \in \{1, \dots,$

$l\}$ as $\mathbf{y}_C(j) = -\frac{1}{d_j} \sum_{i=1}^k \min\{0, n_{i,j}\} (\mathbf{x}(i) - c_i)$. That is, $\mathbf{y}_C(j)$ is the negation of the j 'th coordinate of the output if taking the weighted sum of the inputs on only those coordinates with a negative coefficient $n_{i,j}$. The value $\mathbf{y}_P(j)$ is then similarly defined for all the positive coefficients and the b_j offset: for each $\mathbf{x} \in \text{dom } f$, define $\mathbf{y}_P \in \mathbb{N}^l$ for each $j \in \{1, \dots, l\}$ as

$\mathbf{y}_P(j) = b_j + \frac{1}{d_j} \sum_{i=1}^k \max\{0, n_{i,j}\} (\mathbf{x}(i) - c_i)$. Because $\mathbf{x}(i) - c_i \geq 0$, $\mathbf{y}_P$ and $\mathbf{y}_C$ are always nonnegative. Then if $\mathbf{y} = f(\mathbf{x})$, we have that $\mathbf{y} = \mathbf{y}_P - \mathbf{y}_C$. Define $\hat{f}$ as $\hat{f}(\mathbf{x}) = (\mathbf{y}_P, \mathbf{y}_C)$.

Lemma 4.2

Let $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ be an affine partial function. Then there is a CRC that computes $\hat{f}: \mathbb{N}^k \rightarrow \mathbb{N}^l \times \mathbb{N}^l$ in expected time $O(n \log n)$, where n is the number of input molecules, such that the output molecules monotonically increase with time (i.e. none are ever consumed), and at most $O(n)$ molecules are ever produced.

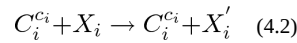
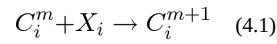
Proof—If $(\mathbf{y}_P, \mathbf{y}_C) = \hat{f}(\mathbf{x})$, then there exist kl integers $n_{1,1}, \dots, n_{k,l} \in \mathbb{Z}$ and $2l + k$ nonnegative integers $b_1, \dots, b_l, c_1, \dots, c_k, d_1, \dots, d_l \in \mathbb{N}$ such that, for each $j \in \{1, \dots, l\}$,

$$\mathbf{y}_C(j) = -\sum_{i=1}^k \frac{1}{d_j} \min\{0, n_{i,j}\} (\mathbf{x}(i) - c_i) \text{ and}$$

$\mathbf{y}_P(j) = b_j + \frac{1}{d_j} \sum_{i=1}^k \max\{0, n_{i,j}\} (\mathbf{x}(i) - c_i)$. Define the CRC as follows. It has input species $\Sigma = \{X_1, \dots, X_k\}$ and output species $\Gamma = \{Y_1^P, \dots, Y_l^P, Y_1^C, \dots, Y_l^C\}$.

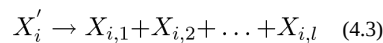
For each $j \in \{1, \dots, l\}$, start with b_j copies of Y_j^P . This accounts for the b_j offsets.

For each $i \in \{1, \dots, k\}$, start with a single molecule C_i^0 , and for each $m \in \{0, \dots, c_i - 1\}$, add the reactions



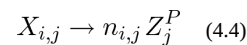
This accounts for the c_i offsets by eventually producing $\mathbf{x}(i) - c_i$ copies of X_i' . Reaction (4.1) takes expected time $O(n)$ to complete because each reaction instance takes expected time at most $O(n)$ (since this is the slowest time for any reaction in volume $O(n)$) and a constant number, c_i , of such reaction instances must take place. Once $C_i^{c_i}$ is produced (hence there are now $\mathbf{x}(i) - c_i$ copies of X_i), reaction (4.2) takes time $O(n \log n)$ to complete by Lemma 2.4.

For each $i \in \{1, \dots, k\}$, add the reaction

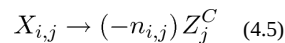


This allows each output to be associated with its own copy of the input. Reaction (4.3) takes time $O(\log n)$ to complete by Lemma 2.3.

For each $i \in \{1, \dots, k\}$ and $j \in \{1, \dots, l\}$, if $n_{i,j} > 0$, add the reaction

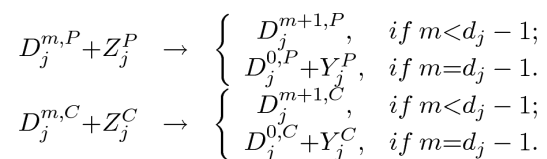


and if $n_{i,j} < 0$, add the reaction



Reaction (4.4) produces $d_j(\mathbf{y}_P(j) - b_j)$ copies of Z_j^P , and reaction (4.5) produces $d_j \mathbf{y}_C(j)$ copies of Z_j^C . Each takes time $O(\log n)$ to complete by Lemma 2.3.

Finally, to produce the correct number of Y_j^P and Y_j^C output molecules, we must divide the count of each Z_j^P and Z_j^C by d_j . For each $j \in \{1, \dots, l\}$, start with a single copy of a molecule $D_j^{0,P}$ and another $D_j^{0,C}$. For each $j \in \{1, \dots, l\}$ and each $m \in \{0, \dots, d_j - 1\}$, add the reactions



By Lemma 2.4, each of these reactions requires time $O(n \log n)$ to complete.

The next lemma characterizes semilinear functions as finite piecewise linear functions, where each of the pieces is defined over an input domain that is a linear set. This will enable us to use CRCs as constructed in Lemma 4.2 to compute semilinear functions in Lemma 4.4.

Lemma 4.3

Let $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ be a semilinear function. Then there is a finite set $\{f_1: \mathbb{N}^k \rightarrow \mathbb{N}^l, \dots, f_m: \mathbb{N}^k \rightarrow \mathbb{N}^l\}$ of affine partial functions, where each $\text{dom } f_i$ is a linear set, such that, for each $\mathbf{x} \in \mathbb{N}^k$, if $f_i(\mathbf{x})$ is defined, then $f(\mathbf{x}) = f_i(\mathbf{x})$, and $\bigcup_{i=1}^m \text{dom } f_i = \mathbb{N}^k$.

We split the semilinear function into partial functions, each with a graph that is a linear set. The non-trivial aspect of our argument is showing that (straightforward) linear algebra over the reals can be used to solve our problem about integer arithmetic. For example, consider a partial function defined by the following linear graph: $\mathbf{b} = \mathbf{0}$, $\mathbf{u}_1 = (1, 1, 1)$, $\mathbf{u}_2 = (2, 0, 1)$, $\mathbf{u}_3 = (0, 2, 1)$ (where the first two coordinates are inputs and the last coordinate is the output). Note that the set of points where this function is defined is where $x_1 + x_2$ is even. Given an input point $\mathbf{x}$, the natural approach to evaluating the function is to solve for the coefficients n_1, n_2, n_3 such that $\mathbf{x}$ can be expressed as a linear combination of $\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3$ restricted to the first two coordinates. Then the linear combination of the last coordinate of $\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3$ with coefficients n_1, n_2, n_3 would give the output. However, the vectors $\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3$ are not linearly independent (yet this linear set cannot be expressed with less than three basis vectors — illustrating the difference between real spaces and integer-valued linear sets), so there are infinitely many real-valued solutions for the coefficients. We show that $\mathbf{u}_i$ must span a real subspace with at most one output value for any input coordinates. Then we can throw out a vector (say $\mathbf{u}_1$) to obtain a set of linearly independent vectors ($\mathbf{u}_2, \mathbf{u}_3$) and solve for $n_2, n_3 \in \mathbb{R}$, and let $n_1 = 0$. In this example, the resulting partial affine function is $f(x_1, x_2) = (x_1 + x_2)/2$.

Proof of Lemma 4.3—Let $G = \{(\mathbf{x}, \mathbf{y}) \in \mathbb{N}^k \times \mathbb{N}^l \mid f(\mathbf{x}) = \mathbf{y}\}$ be the graph of f . Since G is semilinear, it is a finite union of linear sets $\{L_1, \dots, L_n\}$. It suffices to show that each of these linear sets L_m is the graph of an affine partial function. Since L_m is linear, its projection onto any subset of its coordinates is linear. Therefore $\text{dom } f_m$ (the projection of L_m onto its first k coordinates) is linear.

We consider each output coordinate separately, since if we can show that each $\mathbf{y}(j)$ is an affine function of $\mathbf{x}$, then it follows that $\mathbf{y}$ is an affine function of $\mathbf{x}$. Fix $j \in \{1, \dots, l\}$. Let L'_m be the $(k+1)$ -dimensional projection of L_m onto the coordinates defined by $\mathbf{x}$ and $\mathbf{y}(j)$, which is linear because L_m is. Since L'_m is linear, there exist vectors $\mathbf{b}, \mathbf{u}_1, \dots, \mathbf{u}_p \in \mathbb{N}^{k+1}$ such that $L'_m = \{\mathbf{b} + n_1 \mathbf{u}_1 + \dots + n_p \mathbf{u}_p \mid n_1, \dots, n_p \in \mathbb{N}\}$.

Consider the real-vector subspace spanned by $\mathbf{u}_1, \dots, \mathbf{u}_p$. It cannot contain the vector $\mathbf{j} = (0, \dots, 0, 1)^T$. Suppose it does. Take a subset of linearly independent vectors spanning this subspace from the above list (we possibly remove some linearly dependent vectors); say $\mathbf{u}_1, \dots, \mathbf{u}_p$. The unique solution to the coefficients $\xi_1, \dots, \xi_p \in \mathbb{R}$ such that $\mathbf{j} = \xi_1 \mathbf{u}_1 + \dots +$

$\xi_p \mathbf{u}_p'$ can be obtained by using the left-inverse of the matrix with columns $\mathbf{u}_1, \dots, \mathbf{u}_p'$ (the left inverse exists because the matrix is full-rank). Since the elements of the left-inverse matrix are rational functions of the matrix elements, and vectors $\mathbf{u}_1, \dots, \mathbf{u}_p'$ consist of numbers in $\mathbb{N}$, the coefficients $\xi_1, \dots, \xi_p'$ are rational. We can multiply all the coefficients by the least common multiple of their denominators c yielding $c\mathbf{j} = m_1\mathbf{u}_1 + \dots + m_p\mathbf{u}_p'$ where $m_1, \dots, m_p' \in \mathbb{Z}$. Now consider a point $\mathbf{a}$ in L'_m defined as $\mathbf{b} + n_1\mathbf{u}_1 + \dots + n_p\mathbf{u}_p'$ where $n_i \in \mathbb{N}$. Define $n'_i = n_i + m_i$. We choose $\mathbf{a}$ such that n_i are large enough that $n'_i \geq 0$. Since $n'_i \in \mathbb{N}$, we have that both $\mathbf{a}$ and $\mathbf{a} + c\mathbf{j} = \mathbf{b} + n'_1\mathbf{u}_1 + \dots + n'_p\mathbf{u}_p'$ are in L'_m . This is a contradiction because L'_m is the graph of a partial function and cannot contain two different points that agree on their first k coordinates. Therefore $\mathbf{j}$ is not contained in the span of $\mathbf{u}_1, \dots, \mathbf{u}_p$.

Consider again the real-vector subspace spanned by $\mathbf{u}_1, \dots, \mathbf{u}_p$. Again, let $\mathbf{u}_1, \dots, \mathbf{u}_p'$ be a subset of linearly independent vectors spanning this subspace. Since $\mathbf{j}$ is not in it, the subspace must be at most k dimensional. If it is strictly less than k dimensional, add enough vectors in $\mathbb{N}^{k+1}$ to the basis set for the spanned subspace to be exactly k -dimensional but not include $\mathbf{j}$. Call this new set of k linearly independent vectors $\mathbf{w}_1, \dots, \mathbf{w}_k$, where $\mathbf{w}_i = \mathbf{u}_i$ for $i \in \{1, \dots, p\}$. Let $\mathbf{v}_1, \dots, \mathbf{v}_k \in \mathbb{N}^k$ be $\mathbf{w}_1, \dots, \mathbf{w}_k$ restricted to the first k coordinates. The fact that $\mathbf{w}_1, \dots, \mathbf{w}_k$ are linearly independent, but $\mathbf{j}$ is not in the subspace spanned by them, implies that $\mathbf{v}_1, \dots, \mathbf{v}_k$ are linearly independent as well. This can be seen as follows. If $\mathbf{v}_1, \dots, \mathbf{v}_k$ were not linearly independent, then we could write $\mathbf{v}_k = \xi_1\mathbf{v}_1 + \dots + \xi_{k-1}\mathbf{v}_{k-1}$ for some $\xi_i \in \mathbb{R}$. However, $\mathbf{w}_k \neq \mathbf{w}'_k \triangleq \xi_1\mathbf{w}_1 + \dots + \xi_{k-1}\mathbf{w}_{k-1}$. Since $\mathbf{j}$ is proportional to $\mathbf{w}'_k - \mathbf{w}_k$, we obtain a contradiction. Therefore $\mathbf{v}_1, \dots, \mathbf{v}_k$ are linearly independent.

We now describe how to construct an affine function $\mathbf{y}(\mathbf{j}) = f(\mathbf{x})$ for L'_m from $\mathbf{w}_1, \dots, \mathbf{w}_k$. Let matrix $\mathbf{V}$ be the square matrix with $\mathbf{v}_1, \dots, \mathbf{v}_k$ as columns. Let $\mathbf{b}'$ be $\mathbf{b}$ restricted to its first k coordinates. We claim that $\mathbf{y}(\mathbf{j}) = \mathbf{b}(k+1) + (\mathbf{w}_1(k+1), \dots, \mathbf{w}_k(k+1)) \cdot \mathbf{V}^{-1} \cdot (\mathbf{x} - \mathbf{b}')$. Below we'll show that this expression computes the correct value $\mathbf{y}(\mathbf{j})$. But first we show that it defines a partial affine function $f(\mathbf{x})$. Because $\mathbf{v}_1, \dots, \mathbf{v}_k$ are linearly independent, the

inverse $\mathbf{V}^{-1}$ is well-defined. We need to show $f(\mathbf{x}) = b_j + \frac{1}{d_j} \sum_{i=1}^k n_{i,j} (\mathbf{x}(i) - c_i)$ for integer $n_{i,j}$ and nonnegative integer b_j, c_i , and d_j , and that on the domain of f , $\mathbf{x}(i) - c_i \geq 0$. The offset $b_j = \mathbf{b}(k+1)$, which is a non-negative integer because $\mathbf{b}$ is a vector of non-negative integers. Since the offset vector $\mathbf{b}'$ is the same for each output dimension, and it is likewise non-negative, we obtain the offset $c_i = \mathbf{b}'(i)$. Further, since $\mathbf{V}^{-1}$ consists of rational elements (because $\mathbf{V}$ consists of elements in $\mathbb{N}$), we can define d_j and $n_{i,j}$ as needed. Finally, note that the least value of $\mathbf{x}(i)$ that could be in L'_m is $\mathbf{b}'(i) = c_i$, and thus on the domain of f , $\mathbf{x}(i) - c_i \geq 0$.

Finally, we show that this expression computes the correct value $\mathbf{y}(\mathbf{j})$. Let $(\xi_1, \dots, \xi_k)^T \triangleq \mathbf{V}^{-1} \cdot (\mathbf{x} - \mathbf{b}')$, which implies that $\mathbf{x} = \mathbf{b}' + \sum_{i=1}^k \xi_i \mathbf{v}_i$. If our value of $\mathbf{y}(\mathbf{j})$ is incorrect, then $\exists n_1, \dots, n_p \in \mathbb{N}$ such that $\mathbf{b} + \sum_{i=1}^p n_i \mathbf{u}_i$ and $\mathbf{b} + \sum_{i=1}^k \xi_i \mathbf{w}_i$ agree on the first k coordinates but not on the $k+1$ st. Recall that the real-vector subspace spanned by $\mathbf{w}_1, \dots, \mathbf{w}_k$ includes

the subspace spanned by $\mathbf{u}_1, \dots, \mathbf{u}_p$ but does not include $\mathbf{j}$. But $\sum_{i=1}^p n_i \mathbf{u}_i - \sum_{i=1}^k \xi_i \mathbf{w}_i$ is proportional to $\mathbf{j}$ and lies in the subspace spanned by $\mathbf{w}_1, \dots, \mathbf{w}_k$. Therefore we obtain a contradiction, implying that our value of $\mathbf{y}(j)$ is computed correctly.

The next lemma shows that every semilinear function f can be computed by a CRC in $O(n \log n)$ time. It uses a systematic construction based on breaking down f into a finite number of partial affine functions $f_1, \dots, f_m$, in which deciding which f_i to apply is itself a semilinear predicate. Intuitively, the construction proceeds by running many CRCs and CRDs in parallel on input $\mathbf{x}$, computing all f_i 's and all predicates of the form $\varphi_i = \mathbf{x} \in \text{dom } f_i$." The φ_i predicate computation is used to activate (in the case of a "true" answer) or deactivate (in case of "false") the outputs of f_i . Since eventually one CRD stabilizes to "true" and the remainder to "false", eventually the outputs of one f_i are activated and the remainder deactivated, so that the value $f(\mathbf{x})$ is properly computed.

Lemma 4.4

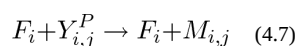
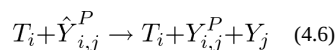
Let $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ be semilinear. Then there is a CRC C that stably computes f , and the expected time for C to reach a count-stable configuration on input $\mathbf{x}$ is $O(n \log n)$, where n is the number of input molecules (the $O()$ constant depends on f but not on n).

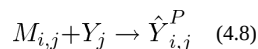
Proof—The CRC will have input species $\Sigma = \{X_1, \dots, X_k\}$ and output species $\Gamma = \{Y_1, \dots, Y_l\}$.

By Lemma 4.3, there is a finite set $\{f_1: \mathbb{N}^k \rightarrow \mathbb{N}^l, \dots, f_m: \mathbb{N}^k \rightarrow \mathbb{N}^l\}$ of affine partial functions, where each $\text{dom } f_i$ is a linear set, such that, for each $\mathbf{x} \in \mathbb{N}^k$, if $f_i(\mathbf{x})$ is defined, then $f(\mathbf{x}) = f_i(\mathbf{x})$. We compute f on input $\mathbf{x}$ as follows. Since each $\text{dom } f_i$ is a linear (and therefore semilinear) set, we compute each predicate $\varphi_i = \mathbf{x} \in \text{dom } f_i$ and $(\forall i' \in \{1, \dots, i-1\}) \mathbf{x} \notin \text{dom } f_{i'}$ by separate parallel CRD's. The latter condition ensures that for each $\mathbf{x}$, precisely one of the predicates is true, in case the domains of the partial functions have nonempty intersection.

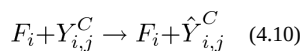
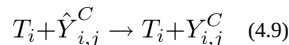
By Lemma 4.2, we can compute each $\hat{f}_i$ by parallel CRC's. Assume that for each $i \in \{1, \dots, m\}$ and each $j \in \{1, \dots, l\}$, the j th pair of outputs $\mathbf{y}_P(j)$ and $\mathbf{y}_C(j)$ of the i th function is represented by species $\hat{Y}_{i,j}^P$ and $\hat{Y}_{i,j}^C$. We interpret each $\hat{Y}_{i,j}^P$ and $\hat{Y}_{i,j}^C$ as an "inactive" version of "active" output species $Y_{i,j}^P$ and $Y_{i,j}^C$.

For each $i \in \{1, \dots, m\}$, we assume that the CRD computing the predicate φ_i represents its output by voting species T_i to represent "true" and F_i to represent "false". Then add the following reactions for each $i \in \{1, \dots, m\}$ and each $j \in \{1, \dots, l\}$:

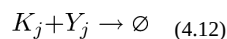
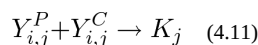




The latter two reactions implement the reverse direction of the first reaction using only bimolecular reactions. Also add the reactions



and



That is, a “true” answer for function i activates the i th output and a “false” answer deactivates the i th output. Eventually each CRD stabilizes so that precisely one i has T_i present, and for all $i' \neq i$, $F_{i'}$ is present. At this point, all outputs for the correct function $\hat{f}_i$ are activated and all other outputs are deactivated. The reactions enforce that at any time,

$\#Y_j = \#K_j + \sum_{i=1}^m (\#Y_{i,j}^P + \#M_{i,j})$. In particular, $\#Y_j \geq \#K_j$ and $\#Y_j \geq \#M_{i,j}$ at all times, so there will never be a K_j or $M_{i,j}$ molecule that cannot participate in the reaction of which it is a reactant. Eventually $\#Y_{i,j}^P$ and $\#Y_{i,j}^C$ stabilize to 0 for all but one value of i (by reaction (4.10)), and for this value of i , $\#Y_{i,j}^P$ stabilizes to $\mathbf{y}(j)$ and $\#Y_{i,j}^C$ stabilizes to 0 (by reaction (4.11)). Eventually $\#K_j$ stabilizes to 0 by reaction (4.12). Eventually $\#M_{i,j}$ stabilizes to 0 since F_i is absent for the correct function $\hat{f}_i$. This ensures that $\#Y_j$ stabilizes to $\mathbf{y}(j)$.

It remains to analyze the expected time to stabilization. Recall $n = \|\mathbf{x}\|$. By Lemma 4.2, the expected time for each affine function computation to complete is $O(n \log n)$. Since we have m parallel computations, and m depends on f but not n , the expected time for all of the computations to complete is $O(n \log n)$. We must also wait for each predicate computation to complete. By Theorem 2.1, each of these predicates takes expected time at most $O(n \log n)$ to complete, so again all of them complete in expected time $O(n \log n)$.

Eventually, the T_i leaders must convert inactive output species to active, and $F_{i'}$ (for $i' \neq i$) must convert active output species to inactive. By Lemma 2.4, each of these requires at most $O(n \log n)$ expected time, and therefore they all complete in expected time at most $O(n \log n)$. Finally, reactions (4.11) and (4.12) are at least as fast as the process described in Lemma 2.4. Thus it takes $O(n \log n)$ expected time for reactions (4.11) and (4.12) to consume all $Y_{i,j}^C$ and K_j molecules, at which point the system has stabilized.

5 Optimization to polylog($\|x\|$) time

Angluin, Aspnes, and Eisenstat combined the slow, deterministic predicate-deciding results of [2] with a fast, error-prone simulation of a bounded-space Turing machine to show that semilinear predicates can be computed without error in expected polylogarithmic time [3]. We show that a similar technique implies that semilinear functions can be computed by CRNs without error in expected polylogarithmic time in the kinetic model, combining the same Turing machine simulation with our $O(n \log n)$ construction described in Lemma 4.4.

We in fact use the construction of [3] in order to conduct the fast, error-prone computation in our proof of Theorem 5.2. The next theorem formalizes the properties of that construction that we require.

Theorem 5.1 ([3])

Let $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ be a function by a $t(m)$ -time-bounded, $s(m)$ -space-bounded Turing machine, where $m \approx \log n$ is the input length in binary, and let $c \in \mathbb{N}$. Then there is a CRC C that computes f correctly with probability at least $1 - n^{-c}$, and the expected time for C to reach a count-stable configuration is $O(t(m)^5)$. Furthermore, the total molecular count never exceeds $O(2^{s(m)})$.

Semilinear functions on an m -bit input can be computed in time $O(m)$ and space $O(m)$ on a Turing machine. Therefore the bounds on CRC expected time and molecular count stated in Theorem 5.1 are $O(\log^5 n)$ and $O(n)$, respectively, when expressed in terms of the number of input molecules n .

Theorem 5.1 is a rephrasing of the main result of Angluin, Aspnes, and Eisenstat [3]. However, a modification of their construction is required to achieve “uniformity” with respect to input size. Rephrasing their construction to our language of CRNs, they allow a different amount of “fuel” species (call it F) for every input size. Indeed, because their model exclusively uses two-reactant, two-product reactions, and thus preserves the total molecular count, this non-uniformity is necessary: the required amount of fuel molecules depends on the space usage $s(m)$, so that the tape of the Turing machine can be accurately represented throughout the computation. We, however, require a uniform initial state. Luckily, we do not need to supply these fuel molecules as part of the input configuration. Instead, these fuels may be generated from the inputs by letting the first reaction of the input X_i be $X_i \rightarrow X'_i + cF$, where X'_i is the input interacting with the rest of the CRC, and $c \in \mathbb{N}$ is chosen large enough that the CRC of [3] will not run out of F molecules. Since the CRC of [3] is used here only to compute semilinear functions, which require only $O(n)$ space to compute on a Turing machine, $c \cdot n$ copies of F are sufficient to run this CRC if c is sufficiently large.

The following theorem is the main theorem of this section.

Theorem 5.2

Let $f: \mathbb{N}^k \rightarrow \mathbb{N}^l$ be semilinear. Then there is a CRC C that stably computes f , and the expected time for C to reach a count-stable configuration is $O(\log^5 n)$, where n is the number of input molecules.

Proof—Our CRC will use the counts of Y_j for each output dimension $\mathbf{y}(j)$ as the global output, and begins by running in parallel:

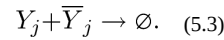
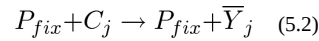
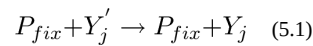
1. A fast, error-prone CRC F to compute $\mathbf{y} = f(\mathbf{x})$ with high probability, as in Theorem 5.1. For any constant $c > 0$, we can design F so that it is correct and finishes in time $O(\log^5 n)$ with probability at least $1 - n^{-c}$, while never reaching total molecular count higher than $O(n)$. We modify the CRC to store the output in three separate sets of species Y_j (the global output), B_j , and C_j redundantly (i.e. $\mathbf{y} = \mathbf{b} = \mathbf{c}$) as follows. Upon halting F copies an “internal” output species $\hat{Y}_j$ to Y_j , B_j , and C_j through reactions $H + \hat{Y}_j \rightarrow H + Y_j + B_j + C_j$ (in asymptotically negligible time).¹⁰ In this way we are guaranteed that the amount of Y_j produced by C is the same as the amounts of B_j and C_j no matter whether its computation is correct or not. This redundant storage is used for later comparison and possible replacement with the slow, deterministic CRC (described next).
2. A slow, deterministic CRC S for $\mathbf{y}' = f(\mathbf{x})$. It is constructed as in Lemma 4.4, running in expected $O(n \log n)$ time.
3. A slow, deterministic CRD D for the semilinear predicate “ $\mathbf{b} = f(\mathbf{x})$?”. It is constructed as in Theorem 2.1 and runs in expected $O(n)$ time.

Following Angluin, Aspnes, and Eisenstat [3], we construct a “timed trigger” as follows, using a single leader molecule, a single “marker” molecule, and $n = \|\mathbf{x}\|$ “interfering” molecules. To ensure that there are always n interfering molecules, we can let them be the input molecules, and a special species I that is generated in the reactions $X_i \rightarrow I + X'_i$, where X'_i is the input species interacting with the remainder of the CRC. The leader will then interact with both X_i and I as interfering molecules.

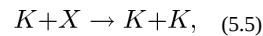
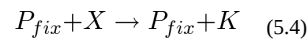
The leader fires the trigger if it encounters the marker molecule, M , d times without any intervening reactions with the interfering molecules, where d is a constant. Note that choosing d larger increases the expected time for this event to happen, since it becomes more likely that the leader encounters an interfering molecule before encountering the M molecule d times in a row. This happens rarely enough that with high probability the trigger fires after F and D finishes (time analysis is presented below). When the trigger fires, it checks if D is outputting a “no” (e.g. has a molecule of L_0), and if so, produces a molecule of P_{fix} . This indicates that the output of the fast CRC F is not to be trusted, and the system should switch from the possible erroneous result of F to the sure-to-be correct result of S .

¹⁰Here, H is some species that is guaranteed with high probability to be absent until F has halted, and then to increase to large ($\Omega(n)$) count in asymptotically negligible time.

Once a P_{fix} is produced, the system converts the output molecules Y'_j of the slow, deterministic CRC S to the global output Y_j , and kills enough of the global output molecules to remove the ones produced by the fast, error-prone CRC:



Finally, P_{fix} triggers a process consuming all species of F other than Y_j , B_j , and C_j in expected $O(\log n)$ time so that afterward, F cannot produce any output molecules. More formally, let Q_F be the set of all species used by F . For all $X \in Q_F \setminus \bigcup_{j=1}^l \{Y_j, B_j, C_j\}$, add the reactions



where $K \notin Q_F$ is a unique species.

First, observe that the output will always eventually converge to the right answer, no matter what happens: If P_{fix} is eventually produced, then the output will eventually be exactly that given by S which is guaranteed to converge correctly. If P_{fix} is never produced, then the fast, error-prone CRC must produce the correct amount of Y_j — otherwise, D will detect a problem.

For the expected time analysis, let us first analyze the trigger. The probability that the trigger leader will fire on any particular reaction number is at most n^{-d} . In time n^2 , the expected number of leader reactions is $O(n^2)$. Thus, the expected number of firings of the trigger in n^2 time is n^{-d+2} . This implies that the probability that the trigger fires before n^2 time is at most n^{-d+2} . The expected time for the trigger to fire is $O(n^d)$.

We now consider the contribution to the total expected time from 3 cases:

1. F is correct, and the trigger fires after time n^2 . There are two subcases: (a) F finishes before the trigger fires. Conditional on this, the whole system converges to the correct answer, never to change it again, in expected time $O(\log^5 n)$. This subcase contributes at most $O(\log^5 n)$ to the total expected time. (b) F finishes after the trigger fires. In this case, we may produce a P_{fix} molecule and have to rely on the slow CRC S . The probability of this case happening is at most n^{-c} . Conditional on this case, the expected time for the trigger to fire is still $O(n^d)$. The whole system converges to the correct answer in expected time $O(n^d)$, because everything

else is asymptotically negligible. Thus the contribution of this subcase to the total expectation is at most $O(n^{-c} \cdot n^d) = O(n^{-c+d})$.

2. F is correct, but the trigger fires before n^2 time. In this case, we may produce a P_{fix} molecule and have to rely on the slow CRC S for the output. The probability of this case occurring is at most n^{-d+2} . Conditional on this case occurring, the expected time for the whole system to converge to the correct answer can be bounded by $O(n^2)$. Thus the contribution of this subcase to the total expectation is at most $O(n^{-d+2} \cdot n^2) = O(n^{-d+4})$.
3. F fails. In this case we'll have to rely on the slow CRC S for the output again. Since this occurs with probability at most n^{-c} , and the conditional expected time for the whole system to converge to the correct answer can be bounded by $O(n^d)$ again, the contribution of this subcase to the total expectation is at most $O(n^{-c} \cdot n^d) = O(n^{-c+d})$.

So the total expected time is bounded by $O(\log^5 n) + O(n^{-c+d}) + O(n^{-d+4}) + O(n^{-c+d}) = O(\log^5 n)$ for $d > 4$, $c > d$.

6 Conclusion

We defined deterministic computation of CRNs corresponding to the intuitive notion that certain systems are guaranteed to converge to the correct answer no matter what order the reactions happen to occur in. We showed that this kind of computation corresponds exactly to the class of functions with semilinear graphs. We further showed that all functions in this class can be computed efficiently.

A work on chemical computation can stumble by attempting to shoehorn an ill-fitting computational paradigm into chemistry. While our systematic construction may seem complex, we are inspired by examples like those shown in Fig. 1 that appear to be good fits to the computational substrate. While delineation of computation that is “natural” for a chemical system is necessarily imprecise and speculative, it is examples such as these that makes us satisfied that we are studying a form of natural chemical computation.

In theoretical computer science, the notion of randomized computation has received significant attention. However, the additional computational power given by error-prone computation compared with deterministic computation is usually rather limited. For example, the class of languages decided by Turing machines, whether they are required to be deterministic or randomized (or even nondeterministic), is the same. In the case of polynomial-time Turing machines, it is widely conjectured [19] that $P = BPP$, i.e., that randomization adds at most a polynomial speedup to any predicate computation. In contrast, CRNs are unusual in the large gap between the power of randomized and deterministic computation: While randomized CRNs can simulate arbitrary Turing machines with high probability [24], deterministic computation is severely limited to semilinear functions only.

Our systematic constructions (unlike the examples in Fig. 1) rely on a carefully chosen initial context — the “extra” molecules that are necessary for the computation to proceed. Some of these species need to be present in a single copy (“leader”). We left unanswered

whether it may be possible to dispense with this level of control of the chemical environment, but this question has since been answered affirmatively by Doty and Hajiaghayi [12]. However, the construction of [12] runs in expected time $O(n)$; it remains open whether there is a leaderless CRNs computing any semilinear function in sublinear expected time.

In contrast to the CRN model discussed in this paper, which is appropriate for small chemical systems in which every single molecule matters, classical “Avogadro-scale” chemistry is modeled using real-valued concentrations that evolve according to mass-action ODEs. Moreover, despite relatively small molecular counts, many biological chemical systems are well-modeled by mass-action ODEs. While the scaling of stochastic CRNs to mass-action systems is understood from a dynamical systems perspective [17], little work has been done comparing their computational abilities. There are hints that single/few-molecule CRNs perform a fundamentally different kind of computation. For example, recent theoretical work has investigated whether CRNs can tolerate multiple copies of the network running in parallel finding that they can lose their computational abilities [9, 10].

Does our notion of deterministic computation have an equivalent in mass-action systems? Consider what happens when the CRN shown in Fig. 1(c) is viewed as a mass-action reaction network, with (non-negative) real-valued inputs $[X_1]_0$, $[X_2]_0$ and output $[Y]_\infty$ (where we use the standard mass-action convention: $[\cdot]_0$ for the initial concentration, and $[\cdot]_\infty$ for the equilibrium concentration). In the limit $t \rightarrow \infty$, the mass-action system will converge to the correct output amount of $[Y]_\infty = \max([X_1]_0, [X_2]_0)$, and moreover, the output amount is independent of what (non-zero) rate constants are assigned to the reactions. Thus one is tempted to connect the notion of deterministic computation studied here and the property of robustness to rate parameters of a mass-action system. Parameter robustness is a recurring motif in biologically relevant reaction networks due to much evidence that biological systems tend to be robust to parameters [5].

However, the connection is not simple. Consider the CRN shown in Fig. 1(a). In the mass-action limit it loses the ability of computing the floor function, but still computes $[Y]_\infty = [X]_0/2$ for real valued $[X]_0$, $[Y]_\infty$, independent of reaction rates. More interestingly, the CRN shown in Fig. 1(b), when considered as a mass-action reaction network, could converge to a different amount of Y as $t \rightarrow \infty$, depending on the rate constants of the last two reactions and the input amounts. Specifically, let k_1 , k_2 , and k_3 be the rate constants of the three reactions, respectively. If $[X_1]_0 > [X_2]_0$ and $k_2 \leq k_3[X_2]_0/([X_1]_0 - [X_2]_0)$, then Y will go to $k_2/k_3([X_1]_0 - [X_2]_0)$ rather than $[X_2]_0$ as in Fig. 1(b). In all other cases, the output will correctly match the function in the figure. (This can be verified by determining the steady states of the system and then determining the stability of each one as a function of the initial concentrations and rate constants.) The cause of the disagreement between stochastic and mass-action instances of this CRN can be identified with the “type I” deviant effect demarcated by Samoilov and Arkin [22].

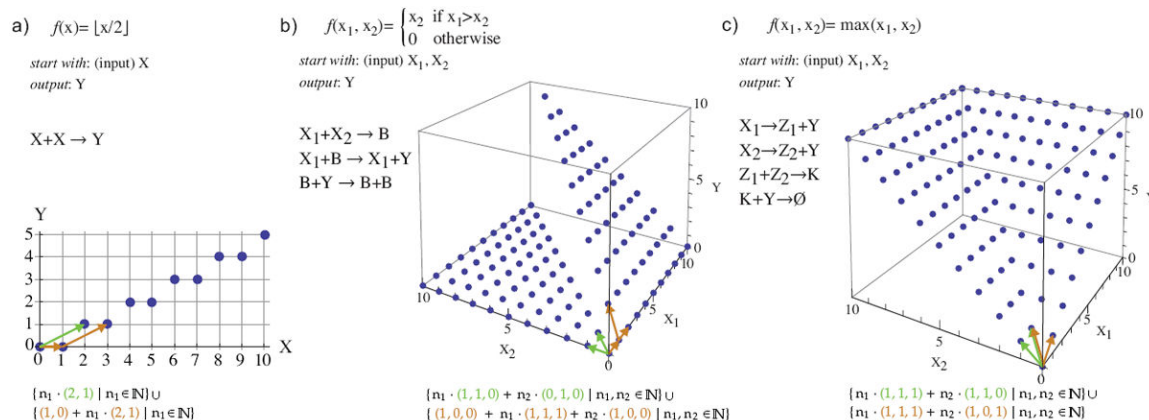
Acknowledgments

We thank Damien Woods and Niranjan Srinivas for many useful discussions, Monir Hajiaghayi for pointing out a problem in an earlier version of this paper, and anonymous reviewers for helpful suggestions.

References

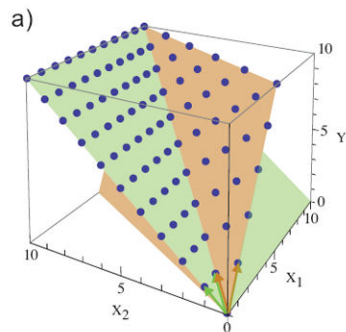
1. Angluin, Dana; Aspnes, James; Diamadi, Zoë; Fischer, Michael; Peralta, René. Computation in networks of passively mobile finite-state sensors. *Distributed Computing*. 2006; 18:235–253. Preliminary version appeared in PODC 2004.
2. Angluin, Dana; Aspnes, James; Eisenstat, David. Stably computable predicates are semilinear. *PODC 2006: Proceedings of the twenty-fifth annual ACM symposium on Principles of distributed computing*; New York, NY, USA: ACM Press; 2006. p. 292-299.
3. Angluin, Dana; Aspnes, James; Eisenstat, David. Fast computation by population protocols with a leader. *Distributed Computing*. Sep; 2008 21(3):183–199. Preliminary version appeared in DISC 2006.
4. Aspnes, James; Ruppert, Eric. An introduction to population protocols. *Bulletin of the European Association for Theoretical Computer Science*. 2007; 93:98–117.
5. Barkai, Naama; Leibler, Stan. Robustness in simple biochemical networks. *Nature*. 1997; 387(6636):913–917. [PubMed: 9202124]
6. Cardelli, Luca. Strand algebras for DNA computing. *Natural Computing*. 2011; 10(1):407–428.
7. Cardelli, Luca; Csikász-Nagy, Attila. The cell cycle switch computes approximate majority. *Scientific Reports*. 2012; 2
8. Chen, Ho-Lin; Doty, David; Soloveichik, David. Deterministic function computation with chemical reaction networks. *DNA 18: Proceedings of The 18th International Meeting on DNA Computing and Molecular Programming, Lecture Notes in Computer Science*; Springer; 2012. p. 25-42.
9. Condon, Anne; Hu, Alan; Mañuch, Ján; Thachuk, Chris. Less haste, less waste: On recycling and its limits in strand displacement systems. *Journal of the Royal Society Interface*. 2012; 2:512–521. Preliminary version appeared in DNA 17.
10. Condon, Anne; Kirkpatrick, Bonnie; Mañuch, Ján. Reachability bounds for chemical reaction networks and strand displacement systems. *DNA 18: 18th International Meeting on DNA Computing and Molecular Programming*; Springer; 2012. p. 43-57.
11. Cook, Matthew; Soloveichik, David; Winfree, Erik; Bruck, Jehoshua. Programmability of chemical reaction networks. In: Condon, Anne; Harel, David; Kok, Joost N.; Salomaa, Arto; Winfree, Erik, editors. *Algorithmic Bioprocesses*. Springer; Berlin Heidelberg; 2009. p. 543-584.
12. Doty, David; Hajiaghayi, Monir. Leaderless deterministic chemical reaction networks. *DNA 2013: Proceedings of the 19th International Meeting on DNA Computing and Molecular Programming*; 2013.
13. Gillespie, Daniel T. Exact stochastic simulation of coupled chemical reactions. *Journal of Physical Chemistry*. 1977; 81(25):2340–2361.
14. Hjelmfelt, Allen; Weinberger, Edward D.; Ross, John. Chemical implementation of neural networks and Turing machines. *Proceedings of the National Academy of Sciences*. 1991; 88(24): 10983–10987.
15. Jiang, Hua; Riedel, Marc; Parhi, Keshab. Digital signal processing with molecular reactions. *IEEE Design and Test of Computers*. 2012; 29(3):21–31.
16. Karp, Richard M.; Miller, Raymond E. Parallel program schemata. *Journal of Computer and system Sciences*. 1969; 3(2):147–195.
17. Kurtz TG. The relationship between stochastic and deterministic models for chemical reactions. *The Journal of Chemical Physics*. 1972; 57(7):2976–2978.
18. Magnasco, Marcelo O. Chemical kinetics is Turing universal. *Physical Review Letters*. 1997; 78(6):1190–1193.
19. Nisan, Noam; Wigderson, Avi. Hardness vs randomness. *Journal of Computer and System Sciences*. 1994; 49(2):149–167.
20. Petri, Carl A. Communication with automata. Technical report, DTIC Document. 1966
21. Presburger, Mojżesz. Über die vollständigkeit eines gewissen systems der arithmetik ganzer zahlen. In welchem die Addition als einzige Operation hervortritt. *Compte Rendus du I Congrès des Mathématiciens des pays Slavs, Warsaw*. 1930:92–101.

22. Samoilov MS, Arkin AP. Deviant effects in molecular reaction pathways. *Nature biotechnology*. 2006; 24(10):1235–1240.
23. Soloveichik, David. Robust stochastic chemical reaction networks and bounded tau-leaping. *Journal of Computational Biology*. 2009; 16(3):501–522. [PubMed: 19254187]
24. Soloveichik, David; Cook, Matthew; Winfree, Erik; Bruck, Jehoshua. Computation with finite stochastic chemical reaction networks. *Natural Computing*. 2008; 7(4):615–633.
25. Soloveichik, David; Seelig, Georg; Winfree, Erik. DNA as a universal substrate for chemical kinetics. *Proceedings of the National Academy of Sciences*. 2010; 107(12):5393.
26. Thachuk, Chris; Condon, Anne. Space and energy efficient computation with DNA strand displacement systems. *DNA 18: Proceedings of The 18th International Meeting on DNA Computing and Molecular Programming*; 2012. p. 135-149.
27. Zavattaro, Gianluigi; Cardelli, Luca. Termination problems in chemical kinetics. *CONCUR 2008-Concurrency Theory*. 2008:477–491.

**Figure 1.**

Examples of deterministically computable functions. (Top) Three functions and examples of CRNs deterministically computing them. The input is represented in the molecular count of X (for (a)), and molecular counts of X_1, X_2 (for (b) and (c)). The output is represented by the molecular count of Y . Example (a) computes via the relative stoichiometry of reactants and products of a single reaction. In example (b), the second and third reactions convert B to Y and vice versa, catalyzed by X_1 and B , respectively. Thus, if there are any X_1 remaining after the first reaction finishes (and thus $x_1 > x_2$), all of B can get converted to Y permanently (since some B is required to convert Y back to B). Since in this case the first reaction produces x_2 molecules of B , x_2 molecules of the output Y are eventually produced. If the first reaction consumes all of X_1 (and thus $x_1 \leq x_2$), eventually any Y that was produced in the second reaction gets converted to B by the third reaction. To see that the CRN in (c) correctly computes the maximum, note that the first two reactions eventually produce $x_1 + x_2$ molecules of Y , while the third reaction eventually produces $\min(x_1, x_2)$ molecules of K . Thus the last reaction eventually consumes $\min(x_1, x_2)$ molecules of Y leaving $x_1 + x_2 - \min(x_1, x_2) = \max(x_1, x_2)$ Y 's. (Bottom) Graphs of the three functions. The set of points belonging to the graph of each of these functions is a semilinear set. Under each plot this semilinear set is written in the form of a union of linear sets corresponding to Equation 1.1. The defining vectors are shown as colored arrows in the graph.

$$f(x_1, x_2) = \max(2x_1 - x_2, x_2)$$



Any semilinear function can be piecewise defined in terms of linear functions, with the decision dictated by semilinear predicates (Lemma 4.3):

$$f(x_1, x_2) = \begin{cases} g(x_1, x_2) = 2x_1 - x_2 & \text{if } x_1 > x_2 \\ h(x_1, x_2) = x_2 & \text{otherwise} \end{cases}$$

b)

start with: (input) X_1, X_2 , (initial context) 1 molecule of F
output: Y

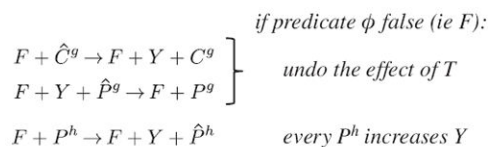
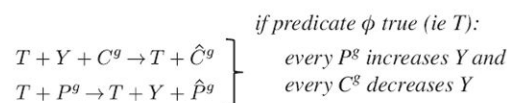
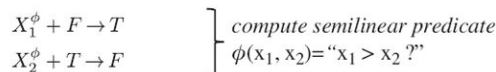
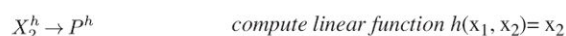
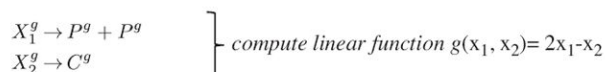
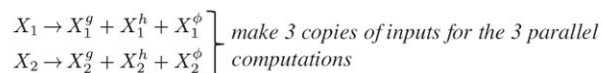


Figure 2.

An example capturing the essential elements of our systematic construction for computing semilinear functions (Lemma 4.4). To compute the target semilinear function, we recast it as a piecewise function defined in terms of linear functions, such that semilinear predicates can decide which of the linear functions is applicable for a given input (this recasting is possible by Lemma 4.3). (a) The graph of the target function visualizing the decomposition into linear functions. (b) A CRN deterministically computing the target function with intuitive explanations of the reactions. We use tri-molecular reactions for simplicity of exposition; however, these can be converted into a sequence of bimolecular reactions. Note that we allow an “initial context”: a fixed set of molecules that are always present in the initial state in addition to the input. The linear functions g and h are computed monotonically by representing the output as the difference of P (“produce”) minus C (“consume”) species. Thus although $P^g - C^g$ could be changing non-monotonically, P^g and C^g do not decrease over time, allowing them to be used as inputs for downstream computation. To compute the semilinear predicate $\phi(x_1, x_2) = “x_1 > x_2?”$, a single molecule, converted between F ($\phi =$ “false”) and T ($\phi =$ “true”) forms, goes back and forth consuming X_1^ϕ and X_2^ϕ . Whether it gets stuck in the F or T forms indicates the excess of X_1^ϕ or X_2^ϕ . The reactions in the right column use the output of this predicate computation to set the count of Y (the global output) to either the value computed by g or h . Note that the CRN cannot “know” when the predicate computation has finished since the absence of X_1^ϕ or X_2^ϕ cannot be detected. Thus the reactions in the right column must be capable of responding to a change in F/T . Species

$\hat{P}^g$, $\hat{P}^h$, and $\hat{C}^g$ are used to backup the values of P^g , P^h , and C^g , enabling the switch in output.